https://pubsonline.informs.org/journal/ijoc

INFORMS JOURNAL ON COMPUTING
Articles in Advance, pp. 1–21
ISSN 1091-9856 (print), ISSN 1526-5528 (online)

Exact MILP Models for Redundancy Allocation with Mixed Components

Jianguang Feng,[a] Zhi-Long Chen,[b] Ada Che,[a,*] Chengbin Chu[c]

[a] School of Management, Northwestern Polytechnical University, Xi'an 710072, China; [b] Robert H. Smith School of Business, University of Maryland, College Park, Maryland 20742; [c] Univ Gustave Eiffel, ESIEE Paris and COSYS-GRETTIA, F-77454 Marne-la-Vallée, France
*Corresponding author

Contact: feng@nwpu.edu.cn, https://orcid.org/0000-0003-4927-9025 (JF); zlchen@umd.edu (Z-LC); ache@nwpu.edu.cn, https://orcid.org/0000-0002-8133-4058 (AC); chengbin.chu@esiee.fr (CC)

Received: July 9, 2024
Revised: February 19, 2025; June 6, 2025; July 5, 2025
Accepted: July 25, 2025
Published Online in Articles in Advance: September 17, 2025

https://doi.org/10.1287/ijoc.2024.0842

Copyright: © 2025 INFORMS

Abstract. The redundancy allocation problem (RAP) aims to find an allocation of redundant components that maximizes system reliability under resource constraints. In this paper, we consider the RAP for complex systems with heterogeneous components, which may have bridges or interconnecting subsystems and are generally more complicated in structure than commonly studied series-parallel systems. A major challenge of the RAP is that the reliability function is nonlinear. To tackle this, we develop six exact mixed integer linear programming (MILP) formulations, which, to our knowledge, are the first of such formulations for the problem. The key idea is to use binary variables and big-M constraints to successively linearize each product term in the reliability function. In addition, we propose a novel tree representation of the reliability function to establish the interconnections between different product terms, which is then used to reduce the number of decision variables and constraints needed in the linearization. We also propose an alternative form of the reliability function that only involves the failure probabilities of subsystems, and leverage this to derive better variable bounds, smaller big-M constants, and simpler formulations. We compare the six MILP formulations by solving them directly using an MILP solver based on three sets of test instances of varying complexity. Results indicate that all formulations achieve optimal solutions quickly for the first set and within reasonable computation times for the second set. For the third set, two formulations can always be solved within a given time limit. Overall, the MILP formulation derived based on the alternative form of the reliability function and a specific type of tree representation scheme performs the best. We also demonstrate that our approach is more robust and scalable than a benchmark branch-and-bound method.

History: Accepted by Pascal Van Hentenryck, Area Editor for Computational Modeling: Methods & Analysis.
Funding: This work was supported by National Natural Science Foundation of China [Grants 72310107003, 72471189, 71901176].
Supplemental Material: The software that supports the findings of this study is available within the paper and its Supplemental Information (https://pubsonline.informs.org/doi/suppl/10.1287/ijoc.2024.0842) as well as from the IJOC GitHub software repository (https://github.com/INFORMSJoC/2024.0842). The complete IJOC Software and Data Repository is available at https://informsjoc.github.io/.

Keywords: redundancy allocation problem • complex systems • mixed integer linear programming • linearization • tree representation

1. Introduction

In system design, using redundant components is one of the main ways to improve system reliability. However, the allocation of redundant components faces various constraints, such as space limitations, weight restrictions, budget considerations, and other technical requirements. One of the widely studied reliability optimization problems is the redundancy allocation problem (RAP), which aims to maximize system reliability by allocating redundant components subject to resource and technical constraints (Coit et al. 1996, Djerdjour and Rekab 2001, Hsieh 2003). As reliability is a strategic issue in many industries such as aerospace, automotive, defense, power, telecommunication, and semiconductor, the RAP has broad applications in system design within these and other industries (Ha and Kuo 2006; Caserta and Voß 2015a, b; Zhao et al. 2015).

The complexity of the RAP depends heavily on the characteristics of the involved system, including the system's structure and component composition. A system typically consists of interconnected subsystems, each

with redundant components such that the subsystem functions if at least one component works. Systems with identical component types in each subsystem are *homogeneous*, while those with varied types are *heterogeneous*. Basic system structures include *series systems*, where subsystems are connected sequentially such that all subsystems must function to make the system operational, and *parallel systems*, where subsystems are arranged in parallel such that at least one subsystem functions to make the system operational. More complicated structures include *series-parallel* (parallel subsystems in series) and *parallel-series* (series subsystems in parallel). A *complex system* features intricate interconnections, such as bridges and interconnecting subsystems, yielding more complicated structures than those in the above-mentioned systems (Ha and Kuo 2006, Park 2020). The most obvious difference between systems is in the form of their reliability when expressed as a function of the number of redundant components. For example, the reliability functions for series-parallel systems are multiplicative across stages and can thus be made separable by (i.e., additive across) stages after taking the logarithm, whereas those for complex systems lack this property.

Most RAP research focuses on deterministic models that aim to maximize reliability subject to linear resource and other constraints, with all problem parameters, including component reliabilities, known with certainty. Other objective functions have also been addressed, such as maximization of the percentile life of systems (Prasad et al. 2001, Coit and Smith 2002, Kim and Kuo 2003), and the simultaneous optimization of reliability and cost (Coit and Konak 2006, Huang et al. 2009). Similar problems but with nonlinear constraints, as well as problems with uncertain component reliability or lifetime, are also studied in the literature (Ha and Kuo 2006, Wang and Li 2020, Li et al. 2023). As an important optimization problem in reliability engineering, the RAP and its numerous variants have received considerable attention from both academic and industrial communities over the last several decades (Gordon 1957, Bellman and Dreyfus 1958, Kuo and Wan 2007, Chang and Kuo 2018). Recent reviews of existing results can be found in Coit and Zio (2019) and Devi et al. (2023).

Chern (1992) shows that the RAP is NP-hard even for series systems with homogeneous components under linear constraints. Consequently, most existing studies on RAP focus on heuristic solution methods (Coit et al. 1996; Coit and Smith 1996; Kulturel-Konak et al. 2003, 2004). Since our focus in this paper is on exact solution methods, we do not review existing studies that focus on heuristics; instead, we refer the reader to the above-mentioned review papers. One of the main difficulties in solving RAP to optimality is that the reliability of any system is a nonlinear function of the number of redundant components to be used in the subsystems, which is nonconcave for most systems (Ha and Kuo 2006). Thus, when it is directly formulated as an optimization problem, the resulting formulation is an integer nonlinear program (INLP), even for systems with a simple structure. Since INLP formulations are among the most difficult optimization problems to solve, it is not a good idea to solve such formulations directly in order to solve RAP to optimality.

Only a few exact algorithms exist for RAP, and most of them are designed for series-parallel systems. Ghare and Taylor (1969) consider the RAP for series-parallel systems with homogeneous components subject to linear constraints. They convert the nonlinear formulation of the problem into a binary integer linear program (BILP), and solve it exactly via branch-and-bound. Their formulation idea is applicable only to the systems they study. When the components are heterogeneous or the system structure changes, their formulation idea no longer works. Bulfin and Liu (1985) study the RAP in systems with homogeneous components for which both the system reliability function and the constraint functions are additive over the stages. One example of such systems is series-parallel systems with homogeneous components with stage-separable resource constraints (which can be nonlinear). They propose two BILP formulations and solve them using branch-and-bound algorithms.

For series-parallel systems with heterogeneous components, Sung and Cho (1999), Caserta and Voß (2015a), and Caserta and Voß (2015b) develop distinct BILP formulations with nonlinear objectives and linear constraints. Sung and Cho (1999) propose a branch-and-bound procedure to solve their formulation; Caserta and Voß (2015a) solve their formulation by a dynamic programming algorithm; Caserta and Voß (2015b) transform their formulation into the multiple-choice knapsack problem (MCKP) and solve it by a branch-and-cut algorithm. Reihaneh et al. (2022) consider a similar problem but under the so-called mixed redundancy strategy, where each subsystem comprises both active and standby components, and their numbers, as well as the sequence of standby components, are all decision variables. They propose a branch-and-price exact algorithm using a column generation framework. This approach requires the objective function to be additive across the stages; hence, it is not applicable to systems without an overall series structure.

The exact algorithms developed in the above-reviewed papers are not applicable to complex systems due to their reliance on certain special structures of the objective function (e.g., being separable across the stages of the system). We are aware of only two studies, Prasad and Kuo (2000) and Ha and Kuo (2006), that develop exact algorithms for the RAP for complex systems with homogeneous components under linear or nonlinear constraints. Both formulates the problem as a general INLP formulation. Prasad and Kuo (2000) propose a

lexicographic search algorithm, essentially an exhaustive enumeration method, whereas Ha and Kuo (2006) propose a branch-and-bound approach based on search space elimination for integer variables. Ha and Kuo (2006) conduct a computational experiment using test instances with up to 25 decision variables generated for two system structures: a bridge system and a hierarchical series-parallel system, both with 10 to 25 subsystems. Their test results show that the branch-and-bound approach requires much less solution time than the lexicographic search algorithm. However, both algorithms exhibit rapid increase of run time with respect to the number of subsystems due to their enumerative nature. Consequently, they are not expected to perform well for complex systems with heterogeneous components, as many more decision variables may be needed to formulate the RAP for such systems.

Finally, it is worth noting that a more recent study by Park (2020) develops an approximate solution method for the RAP for complex systems with heterogeneous components under linear constraints. The approximation method of Park (2020) works as follows. First, a set of random samples is drawn based on the success-failure probability of each component, and for the given set of samples, the RAP is formulated as a mixed integer linear program (MILP). The size of the MILP increases with the sample size. As the sample size increases to infinity, the solution of the MILP converges to the optimal solution. The obvious drawback of this approach is that the solution from the MILP for a given set of samples is not guaranteed to be optimal. To obtain a guaranteed optimal solution, the sample size needs to be infinitely large, rendering the resulting MILP unsolvable.

In this paper, we study the RAP for complex systems with heterogeneous components under linear constraints. We develop several exact MILP formulations for this problem, which, to our knowledge, are the first such formulations for the problem. A major benefit of using MILP formulations is that we can leverage the latest developments in the commercial MIP solver industry and directly apply available MIP solvers to solve the MILP formulations we develop for RAP.

Our key formulation idea is to use binary variables and big-M constraints to successively linearize each product term in the objective function. In addition, we develop several techniques to make the formulations more effective. We propose a tree-based representation of the reliability function, where each product term corresponds to a unique path in the tree, to establish the interconnections between different product terms in the reliability function. By leveraging this structure, we can streamline the linearization process of these product terms, yielding models with fewer decision variables and constraints. We also propose an alternative form of the reliability function which only involves the failure probabilities of subsystems, and leverage this to derive tighter variable bounds, smaller big-M values and simpler formulations. Our computational experiments show that the proposed MILP models can solve all benchmark RAP instances from the literature to optimality within a few seconds, as well as much larger instances—generated based on several large complex systems from existing studies—within a reasonable amount of time. For some benchmark instances, optimal solutions have been reported in the literature without proven optimality. We show their optimality for the first time, which provides a baseline for future research. In addition, we demonstrate that our approach is more robust and scalable than a benchmark branch-and-bound method.

The remainder of this paper is structured as follows. Section 2 defines the problem precisely and introduces the necessary notation. In Section 3, several MILP models for this problem are developed based on the general-form reliability function. In Section 4, computational results are presented. Finally, we conclude the work in Section 5.

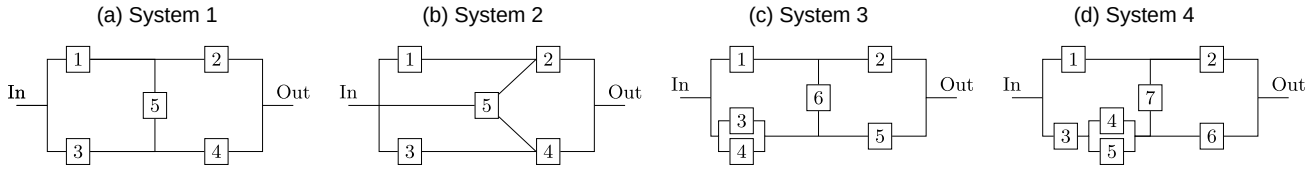
2. Problem Description and Formulation

We first, in Section 2.1, define the redundancy allocation problem in complex systems with heterogeneous components and formulate it as a binary integer nonlinear program (BINLP) with a nonlinear objective function and linear constraints. We then, in Section 2.2, briefly discuss how the reliability of the given system, which is the objective function of the BINLP formulation, can be derived as a function of the reliabilities of subsystems.

2.1. Definition and Formulation

Any system consists of one input and one output interconnected through a set $\mathcal{J} = \{1, 2, \ldots, J\}$ of subsystems. The subsystems are connected following a given network structure. Figure 1 gives four examples of complex systems from the literature (Park 2020), where each square represents a subsystem. Each subsystem $j \in \mathcal{J}$ can be configured with various types of components. Let $\mathcal{H}_j = \{1, 2, \ldots, H_j\}$ denote the set of available component types that can be used for subsystem $j \in \mathcal{J}$. The reliability of a type-h component in subsystem $j \in \mathcal{J}$ is r_{jh}, and its failure probability is $q_{jh} = 1 - r_{jh}$. As commonly assumed in the RAP literature, the component failures are independent, and different components of the same type have the same reliability. The number of configured

Figure 1. Examples of Complex Systems



components of each type $h \in \mathcal{H}_j$ is a decision variable to be determined. A subsystem is deemed functional if at least one component within that subsystem is functional. Similarly, the entire system is functional if there exists a path from the input to the output along which all subsystems involved in the path are simultaneously functional.

The RAP is to find the number of components of each type to use in each subsystem to maximize the system's reliability, subject to resources and other constraints as follows. There are I resources available, whose set is denoted as $\mathcal{I} = \{1, 2, \ldots, I\}$. Each type-$h$ component in subsystem j consumes a_{ijh} units of resource i in order to function, for all $i \in \mathcal{I}$, $j \in \mathcal{J}$, and $h \in \mathcal{H}_j$. The total available quantity of resource $i \in \mathcal{I}$ is limited by b_i. In addition, there is an upper bound, denoted by K_{jh}, on the number of type-h components that can be used in subsystem $j \in \mathcal{J}$. For notional convenience, we define $\mathcal{K}_{jh} = \{0, 1, \ldots, K_{jh}\}$.

Let x_{jh} be the nonnegative integer variable denoting the number of type-h components used in subsystem j, for $j \in \mathcal{J}$ and $h \in \mathcal{H}_j$. As each subsystem fails only if all of its components fail (i.e., active redundancy), the reliability of subsystem $j \in \mathcal{J}$ is defined as $R_j(x) = 1 - \prod_{h \in \mathcal{H}_j} (q_{jh}^{x_{jh}})$, and consequently, the probability that subsystem $j \in \mathcal{J}$ fails is $Q_j(x) = 1 - R_j(x) = \prod_{h \in \mathcal{H}_j} (q_{jh}^{x_{jh}})$. When there is no ambiguity, we use R_j and Q_j to denote $R_j(x)$ and $Q_j(x)$, respectively, for notational convenience. The reliability of the system, as a function of the decision variables x and denoted as $f(x)$, is shown in Section 2.2 to be a nonlinear function of $Q_1, \ldots, Q_J$ and $R_1, \ldots, R_J$ that can always be written as an integer constant plus a certain number of product terms with integer coefficients, where each product term is the product of some Q_j's and $R_{j'}$'s with $j \neq j'$, that is, $f(x)$ is in the following form:

$$f(x) = \alpha + \sum_{l \in \mathcal{L}} \beta_l \left(\prod_{j \in \mathcal{J}_l^Q} Q_j \prod_{j' \in \mathcal{J}_l^R} R_{j'} \right), \tag{1}$$

where α is an integer constant, $\mathcal{L} = \{1, 2, \ldots, L\}$ is the index set of all the product terms involved with L being the number of such terms, β_l is an integer coefficient for the lth product term, and $\mathcal{J}_l^Q \subseteq \mathcal{J}$ and $\mathcal{J}_l^R \subseteq \mathcal{J}$ are the index sets of subsystems corresponding to the Q_j's and $R_{j'}$'s in the lth product term, respectively. As shown in Section 2.2, $\mathcal{J}_l^Q$ and $\mathcal{J}_l^R$ do not overlap, and one of them could be empty, for any $l \in \mathcal{L}$. We note that, as discussed in Section 2.2, the specific structure and terms involved in (1) depend on the topology of the system and the specific method used to derive $f(x)$, but for any given system and the specific method used, an exact expression of (1) can be derived. The value of α could be any integer, and β_l's are mostly 1 or -1, but a small number of them could be other integers.

The RAP can then be directly formulated as an INLP as follows.

$$\max \left\{ f(x) \Big| \sum_{j \in \mathcal{J}} \sum_{h \in \mathcal{H}_j} a_{ijh} x_{jh} \leq b_i, \ \forall i \in \mathcal{I}; \sum_{h \in \mathcal{H}_j} x_{jh} \geq 1, x_{jh} \in \mathcal{K}_{jh}, \quad \forall j \in \mathcal{J}, h \in \mathcal{H}_j \right\}. \tag{2}$$

The first constraint requires that the quantity of each type of resource consumed must not exceed the quantity available. The second constraint means that each subsystem must have at least one component. If additional linear constraints are introduced, the formulation remains an INLP with linear constraints.

Due to the nonlinear objective function $f(x)$, it is not practical to solve formulation (2) directly by an optimization solver. In Section 3, we develop techniques to convert this formulation to equivalent MILP formulations, which standard MILP solvers can then solve. To this end, we first need to convert the general integer variables in the formulation (2) into binary integer variables. There are two common methods to accomplish this. The first method is to replace x_{jh} with $K_{jh} + 1$ binary variables x_{jh}^k, for $k = 0, \ldots, K_{jh}$, such that x_{jh}^k takes 1 if and only if x_{jh} takes the value of k. Clearly, for each pair of j and h, exactly one of the binary variables $x_{jh}^0, \ldots, x_{jh}^{K_{jh}}$ should take value 1. Expressed as a function of the binary variables x_{jh}^k, the reliability of subsystem j, $R_j(x)$, and its failure

probability $Q_j(x)$, need to be redefined as

$$Q_j(x) = \prod_{h=1}^{H_j} \left(\sum_{k=0}^{K_{jh}} (q_{jh})^k x_{jh}^k \right), \tag{3}$$

$$R_j(x) = 1 - Q_j(x), \tag{4}$$

and the reliability of the system $f(x)$ is still in the same functional form of $R_1(x), \ldots, R_J(x)$ and $Q_1(x), \ldots, Q_J(x)$, after these Q_j and R_j functions are redefined as above. Using the binary variables x_{jh}^k's defined as above, the formulation (2) can then be converted to the following BINLP:

$$\max \quad f(x) = \alpha + \sum_{l \in \mathscr{L}} \beta_l \left(\prod_{j \in \mathscr{J}_l^Q} Q_j \prod_{j' \in \mathscr{J}_l^R} R_{j'} \right) \tag{5}$$

$$\text{s.t.} \sum_{j \in \mathscr{J}} \sum_{h \in \mathscr{H}_j} \sum_{k \in \mathscr{K}_{jh}} k a_{ijh} x_{jh}^k \le b_i, \quad \forall i \in \mathscr{I}, \tag{6}$$

$$\sum_{k \in \mathscr{K}_{jh}} x_{jh}^k = 1, \quad \forall j \in \mathscr{J}, h \in \mathscr{H}_j, \tag{7}$$

$$\sum_{h \in \mathscr{H}_j} \sum_{k \in \mathscr{K}_{jh} \setminus \{0\}} x_{jh}^k \ge 1, \quad \forall j \in \mathscr{J}, \tag{8}$$

$$x_{jh}^k \in \{0,1\}, \quad \forall j \in \mathscr{J}, h \in \mathscr{H}_j, k \in \mathscr{K}_{jh}. \tag{9}$$

The second method to convert general integer variables x_{jh}'s to binary variables in the formulation (2) is to represent x_{jh} by $\lceil \log_2(K_{jh}+1) \rceil$ binary variables x_{jh}^k, for $k = 1, \ldots, \lceil \log_2(K_{jh}+1) \rceil$ such that x_{jh} can be written as a linear function of these binary variables (Caserta and Voß 2015a). A corresponding BINLP formulation, which is slightly different from (5)–(9), is given in Online Appendix D. All the linearization and other techniques developed later in the paper for the formulation (5)–(9) can be adapted to the formulation given in Online Appendix D to derive corresponding MILP formulations. However, our computational tests show that such resulting MILP formulations do not perform better than the MILP formulations developed for (5)–(9). Thus, in the remainder of the paper, we use the formulation (5)–(9) derived from the first method and develop MILP formulations based on it, and do not discuss how the formulation given in Online Appendix D can be used to build MILP formulations.

2.2. Calculation of the System Reliability

In the literature, there are several different methods for calculating the reliability function $f(x)$ of a given complex system (Rai et al. 1995, Sallak et al. 2009). In the following, we briefly describe the minimal-path method (Lin et al. 1976, Rai et al. 1995). Another two methods, the minimal-cut method (Sallak et al. 2009, Park 2020) and the factoring method (Wood 1985, Rai et al. 1995, Konak and Smith 2011), are presented in Online Appendix A. Using any of these methods, the derived expression of the reliability function $f(x)$ can always be written in the form given in (1).

On the network structure of a given complex system, a *path* is defined as a sequence of subsystems that connects the input and output of the system such that if all these subsystems are functional, then the system is functional. We say a path is operational when all the subsystems on it are functional. A *minimal path* is defined as a path such that any proper subset of the subsystems contained in it does not form a path. For example, System 1 in Figure 1 contains four minimal paths: subsystems 1, 2; subsystems 3, 4; subsystems 1, 4, 5, and subsystems 2, 3, 5. The minimal-path method (Lin et al. 1976, Rai et al. 1995) enumerates all minimal paths from the input to the output and derives the system reliability based on the simple fact that the system is operational if and only if at least one of these minimal paths is operational. In a given system, let N_p be the number of minimal paths, P_i the probability that the ith minimal path is operational, …, and $P_{i_1 \ldots i_u}$, for $u \ge 2$, the probability that the

i_1th, i_2th, …, and i_uth minimal paths are all operational simultaneously. The system's reliability can be calculated as (Lin et al. 1976, Sallak et al. 2009):

$$f(x) = \sum_{i=1}^{N_p} P_i - \sum_{i=1}^{N_p-1} \sum_{j=i+1}^{N_p} P_{ij} + \sum_{i=1}^{N_p-2} \sum_{j=i+1}^{N_p-1} \sum_{k=j+1}^{N_p} P_{ijk} - \cdots + (-1)^{N_p-1} P_{12\ldots N_p}. \tag{10}$$

There are a total of $2^{N_p} - 1$ terms summed up in (10), where each term is the probability that one minimal path is operational or multiple minimal paths are all operational. Let $\mathscr{P}_i$ denote the set of subsystems contained in the ith minimal path. Let R_j denote the probability that subsystem j is operational, and $Q_j = 1 - R_j$ the probability that subsystem j fails. For any $u \geq 1$ and any given combination of u minimal paths $i_1, \ldots, i_u$, the probability that the i_1th, …, and i_uth minimal paths are all operational can be calculated as follows:

$$P_{i_1 \ldots i_u} = \prod_{j \in \mathscr{P}_{i_1} \cup \mathscr{P}_{i_2} \cup \cdots \cup \mathscr{P}_{i_u}} R_j.$$

Clearly, for any subsystem $j \in \mathscr{J}$, R_j appears at most once in the product term given above for $P_{i_1 \ldots i_u}$. Therefore, the system reliability given in (10) can be written as a function of the reliabilities of the subsystems, and this function has the same form as (1), except that every set $\mathscr{J}_l^Q$ is empty.

Each method for calculating the reliability of a system has pros and cons. It can be time-consuming for any method to derive the reliability function $f(x)$ for a large complex system because there are often exponential numbers of minimal paths, minimal cut sets, or smaller systems that need to be considered. Interested readers are referred to the papers cited above as well as more recent studies such as Zhou et al. (2023) for further information. Since this paper does not focus on deriving the reliability of a system, we do not further discuss this topic. Instead, the remainder of the paper focuses on developing efficient MILP models for the RAP described in Section 2, and assumes that the reliability function $f(x)$ is given in the form of (1).

We show in Online Appendix B how the reliability of System 1 in Figure 1 can be derived using different methods. The reliability functions of the four systems in Figure 1 are denoted as $f_1(x), \ldots, f_4(x)$, respectively, and their specific forms derived using the factoring method are given as follows.

$$f_1(x) = Q_1Q_2Q_3Q_4R_5 - Q_1Q_3R_5 - R_1R_2R_3R_4Q_5 + R_1R_2Q_5 - Q_2Q_4R_5 + R_3R_4Q_5 + R_5. \tag{11}$$

$$f_2(x) = R_1R_2R_3R_4Q_5 + R_1R_2Q_5 - Q_2Q_4R_5 + R_3R_4Q_5 + R_5. \tag{12}$$

$$f_3(x) = Q_1Q_2Q_3Q_4Q_5R_6 - Q_1Q_3Q_4R_6 + R_1R_2Q_3Q_4R_5Q_6 - R_1R_2R_5Q_6 + R_1R_2Q_6 - Q_2Q_5R_6 - Q_3Q_4R_5Q_6 + R_5Q_6 + R_6. \tag{13}$$

$$\begin{aligned} f_4(x) = {} & Q_1Q_2Q_3Q_6R_7 + Q_1Q_2R_3Q_4Q_5Q_6R_7 - Q_1Q_3R_7 - Q_1R_3Q_4Q_5R_7 - Q_2Q_6R_7 \\ & - Q_3R_6Q_7 + R_1R_2Q_3R_6Q_7 + R_1R_2R_3Q_4Q_5R_6Q_7 - R_1R_2R_6Q_7 + R_1R_2Q_7 \\ & - R_3Q_4Q_5R_6Q_7 + R_6Q_7 + R_7. \end{aligned} \tag{14}$$

3. Mixed Integer Linear Programming Models

In this section, we reformulate the BINLP formulation (5)–(9) given in Section 2.1 into several MILPs, which can then be solved efficiently by commercial MIP solvers. In Section 3.1, we develop a basic MILP formulation. Then, a modified MILP model is derived based on a tree representation of the reliability function presented in Section 3.2. In Section 3.3, we discuss how we can derive alternative formulations based on the above models by changing the form of the reliability function and making an alternative tree representation of the reliability function.

3.1. Basic MILP Formulation

We first describe the process of linearizing each nonlinear term in the objective function (5) by introducing continuous variables and linear big-M constraints. Once this is done, the BINLP formulation (5)–(9) becomes a MILP formulation. We also discuss how to derive reasonable values for the involved big-M constants in order to make the MILP formulation as efficient as possible.

3.1.1. Linearization of Product Terms. Since each product term in the objective function (5) has the same structure, our following analysis is thus focused on a single product term, $\prod_{j\in\mathscr{J}_l^Q} Q_j \prod_{j'\in\mathscr{J}_l^R} R_{j'}$, for any $l\in\mathscr{L}$, and show how it can be linearized.

Let $\mathscr{J}_l = \mathscr{J}_l^Q \cup \mathscr{J}_l^R$. Since, as discussed in Section 2.1, $\mathscr{J}_l^Q$ and $\mathscr{J}_l^R$ do not overlap, $|\mathscr{J}_l| = |\mathscr{J}_l^Q| + |\mathscr{J}_l^R|$. For notional convenience, we define $J_l = |\mathscr{J}_l|$. Let $\delta(s)\in\mathscr{J}$ denote the subsystem corresponding to the sth factor in the product term, that is, the sth factor is either $Q_{\delta(s)}$ or $R_{\delta(s)}$, for $s = 1,\ldots,J_l$. As an example, the second factor in the product term $Q_1Q_3R_5$ is $Q_{\delta(2)}$ with $\delta(2) = 3$. For ease of presentation, for $s = 1,\ldots,J_l$, we denote the sth factor in the product term as O_s which is $Q_{\delta(s)}$ if $\delta(s)\in\mathscr{J}_l^Q$, and $R_{\delta(s)}$ if $\delta(s)\in\mathscr{J}_l^R$. This enables us to write the product term succinctly as $\prod_{s=1}^{J_l} O_s$. We propose an iterative procedure to linearize this product term as follows: First, linearize O_1; then linearize O_1O_2; …, and finally, linearize $O_1O_2\cdots O_{J_l}$. In the following, we provide a detailed description of how this can be achieved.

We introduce a continuous variable φ_{ls} as the product of the first s factors, that is, $\varphi_{ls} = \prod_{s'=1}^{s} O_{s'}$, for $s = 1,\ldots,J_l$. Let $\varphi_{l0} = 1$ be a given constant. Thus,

$$\varphi_{ls} = \varphi_{l,s-1}O_s,\quad \forall s = 1,\ldots,J_l. \tag{15}$$

Since $\prod_{s=1}^{J_l} O_s = \varphi_{lJ_l}$, if we can linearize each nonlinear constraint in (15), then the product term $\prod_{s=1}^{J_l} O_s$ is linearized. In the following, we discuss how to linearize the nonlinear constraints involved in (15) by introducing additional continuous variables and linear constraints.

By the definition of O_s, (15) can be expanded as

$$\varphi_{ls} = \begin{cases} \varphi_{l,s-1}Q_{\delta(s)}, \text{ if } \delta(s)\in\mathscr{J}_l^Q, \\ \varphi_{l,s-1}R_{\delta(s)} = \varphi_{l,s-1} - \varphi_{l,s-1}Q_{\delta(s)}, \text{ if } \delta(s)\in\mathscr{J}_l^R, \end{cases} \quad \forall s = 1,\ldots,J_l. \tag{16}$$

Hence, it is necessary to linearize the common term $\varphi_{l,s-1}Q_{\delta(s)}$ present in both the cases in (16). By the definition of Q_j given by (3), we have

$$\varphi_{l,s-1}Q_{\delta(s)} = \varphi_{l,s-1}\prod_{h\in\mathscr{H}_{\delta(s)}}\left(\sum_{k\in\mathscr{K}_{\delta(s)h}} (q_{\delta(s)h})^k x^k_{\delta(s)h}\right),\quad \forall s = 1,\ldots,J_l. \tag{17}$$

Thus, to linearize $\varphi_{l,s-1}Q_{\delta(s)}$, we need to linearize the right-hand-side of (17), which is the product of $\varphi_{l,s-1}$ and a product term consisting of multiple factors. To this end, we again use an iterative procedure as follows. We introduce new variables ρ_{lsh}, for $h = 0,1,\ldots,H_{\delta(s)}$, and define $\rho_{ls0} = \varphi_{l,s-1}$, and ρ_{lsh} to be the product of $\varphi_{l,s-1}$ and the first h factors in the product term involved in (17), that is, $\rho_{lsh} = \varphi_{l,s-1}\prod_{h'=1}^{h}(\sum_{k\in\mathscr{K}_{\delta(s)h'}}(q_{\delta(s)h'})^k x^k_{\delta(s)h'})$. Thus,

$$\rho_{lsh} = \rho_{ls,h-1}\left(\sum_{k\in\mathscr{K}_{\delta(s)h}} (q_{\delta(s)h})^k x^k_{\delta(s)h}\right),\quad \forall s = 1,\ldots,J_l, h\in\mathscr{H}_{\delta(s)}. \tag{18}$$

With the variables ρ_{lsh} defined as above, variable $\rho_{lsH_{\delta(s)}}$ represents the right-hand-side of (17). Thus, to linearize the right-hand-side of (17), we need to linearize the nonlinear relation (18). Since exactly one of the binary variables $x^k_{\delta(s)h}$, for $k\in\mathscr{K}_{\delta(s)h}$, is 1, and all others are 0, exactly one of the terms in the summation $\sum_{k\in\mathscr{K}_{\delta(s)h}}(q_{\delta(s)h})^k x^k_{\delta(s)h}$ on the right-hand-side of (18) is nonzero, and all other terms are 0. This means (18) can be reformulated as the following linear inequalities:

$$\rho_{lsh} \le \rho_{ls,h-1}(q_{\delta(s)h})^k + M^{k+}_{lsh}(1 - x^k_{\delta(s)h}),\quad \forall s = 1,\ldots,J_l, h\in\mathscr{H}_{\delta(s)}, k\in\mathscr{K}_{\delta(s)h}, \tag{19}$$

$$\rho_{lsh} \ge \rho_{ls,h-1}(q_{\delta(s)h})^k - M^{k-}_{lsh}(1 - x^k_{\delta(s)h}),\quad \forall s = 1,\ldots,J_l, h\in\mathscr{H}_{\delta(s)}, k\in\mathscr{K}_{\delta(s)h}, \tag{20}$$

where M^{k+}_{lsh} and M^{k-}_{lsh} are sufficiently large positive numbers, also known as big-M constants.

From the above discussion, it can be seen that all the ρ_{lsh} variables, for $h = 0,1,\ldots,H_{\delta(s)}$, and their relationships are completely defined by the linear constraints defined above, including $\rho_{ls0} = \varphi_{l,s-1}$, and (19) and (20). Since, as

discussed above, $\rho_{lsH_{\delta(s)}} = \varphi_{l,s-1}Q_{\delta(s)}$, we can replace the nonlinear term $\varphi_{l,s-1}Q_{\delta(s)}$ in (16) by variable $\rho_{lsH_{\delta(s)}}$. This converts the nonlinear relations in (16) and hence those in (15) into linear relations. Therefore, all the φ_{ls} variables, for $s = 1,\ldots,J_l$, and their relations are completely defined by linear constraints. This achieves our goal of linearizing every product term in the nonlinear reliability function (1).

Now, putting together all the new variables (which are all continuous) and the linear constraints defined above, the BINLP formulation (5)–(9) can be reformulated as an equivalent MILP model:

$$
\begin{aligned}
[\text{BASIC}]\quad \max\ & f(x) = \alpha + \sum_{l\in\mathscr{L}} \beta_l \varphi_{lJ_l} \\
\text{s.t.}\ & \varphi_{l0} = 1, \quad \forall l \in \mathscr{L}, \\
& \rho_{ls0} = \varphi_{l,s-1}, \quad \forall l \in \mathscr{L}, s = 1,2,\ldots,J_l, \\
& \rho_{lsh} \le \rho_{ls,h-1}(q_{\delta(s)h})^k + M^{k+}_{lsh}(1 - x^k_{\delta(s)h}), \quad \forall l \in \mathscr{L}, s = 1,2,\ldots,J_l, h \in \mathscr{H}_{\delta(s)}, k \in \mathscr{K}_{\delta(s)h}, \\
& \rho_{lsh} \ge \rho_{ls,h-1}(q_{\delta(s)h})^k - M^{k-}_{lsh}(1 - x^k_{\delta(s)h}), \quad \forall l \in \mathscr{L}, s = 1,2,\ldots,J_l, h \in \mathscr{H}_{\delta(s)}, k \in \mathscr{K}_{\delta(s)h}, \\
& \varphi_{ls} = \rho_{lsH_{\delta(s)}}, \ \forall l \in \mathscr{L}, s = 1,2,\ldots,J_l, \delta(s) \in \mathscr{J}^Q_l, \\
& \varphi_{ls} = \varphi_{l,s-1} - \rho_{lsH_{\delta(s)}}, \ \forall l \in \mathscr{L}, s = 1,2,\ldots,\mathscr{J}_l, \delta(s) \in \mathscr{J}^R_l, \\
& \sum_{j\in\mathscr{J}} \sum_{h\in\mathscr{H}_j} \sum_{k\in\mathscr{K}_{jh}} k a_{ijh} x^k_{jh} \le b_i, \quad \forall i \in \mathscr{I}, \\
& \sum_{k\in\mathscr{K}_{jh}} x^k_{jh} = 1. \quad \forall j \in \mathscr{J}, \quad h \in \mathscr{H}_j, \\
& \sum_{h\in\mathscr{H}_j} \sum_{k\in\mathscr{K}_{jh}\setminus\{0\}} x^k_{jh} \ge 1, \quad \forall j \in \mathscr{J}, \\
& 0 \le \rho_{lsh} \le 1, \quad \forall l \in \mathscr{L}, \quad s = 1,2,\ldots,J_l, \quad h \in \mathscr{H}_{\delta(s)}, \\
& 0 \le \varphi_{ls} \le 1, \quad \forall l \in \mathscr{L}, \quad s = 1,2,\ldots,J_l, \\
& x^k_{jh} \in \{0,1\} \quad \forall j \in \mathscr{J}, \quad h \in \mathscr{H}_j, \quad k \in \mathscr{K}_{jh}.
\end{aligned}
$$

Note that while the variables φ_{ls}'s are introduced to streamline the linearization procedure, they can be eliminated to simplify the formulation. This can be achieved by removing the constraints involving φ_{lJ_l} for $l \in \mathscr{L}$, replacing variable φ_{ls} with $\rho_{l,s+1,0}$ for $l \in \mathscr{L}, s = 1,\ldots,J_l - 1$ in the constraints, and replacing φ_{lJ_l} with $\rho_{lJ_lH_{\delta(J_l)}}$ if $\delta(J_l) \in \mathscr{J}^Q_l$ and $\rho_{lJ_l0} - \rho_{lJ_lH_{\delta(J_l)}}$ if $\delta(J_l) \in \mathscr{J}^R_l$ in the objective function. Specifically, we substitute the constraints involving φ_{ls} in model [BASIC] with

$$
\begin{aligned}
& \rho_{l10} = 1, \\
& \rho_{ls0} = \rho_{l,s-1,H_{\delta(s)}}, \ \forall l \in \mathscr{L}, s = 2,\ldots,J_l, \delta(s) \in \mathscr{J}^Q_l, \\
& \rho_{ls0} = \rho_{l,s-1,0} - \rho_{l,s-1,H_{\delta(s)}}, \ \forall l \in \mathscr{L}, s = 2,\ldots,J_l, \delta(s) \in \mathscr{J}^R_l,
\end{aligned}
$$

and replace the objective function with

$$
f(x) = \alpha + \sum_{l\in\mathscr{L}, \delta(J_l)\in\mathscr{J}^Q_l} \beta_l \rho_{lJ_lH_{\delta(J_l)}} + \sum_{l\in\mathscr{L}, \delta(J_l)\in\mathscr{J}^R_l} \beta_l (\rho_{lJ_l0} - \rho_{lJ_lH_{\delta(J_l)}}).
$$

However, our computational tests show that this transformation does not necessarily make the model easier or more difficult to solve. Therefore, we opt to retain the variables φ_{ls}'s and the corresponding constraints in the formulation.

3.1.2. Determination of Big-M Constants. The big-M constants M^{k+}_{lsh}'s and M^{k-}_{lsh}'s should be sufficiently large to ensure that the resulting linear constraints (19) and (20) are equivalent to the original nonlinear one (18). By the definitions of ρ_{lsh}'s and x^k_{jh}'s, an evident choice for these constants is 1. However, overly large values may result in poor linear relaxation. To address this concern, we need to derive tight values for these constants. To derive a

proper value for M_{lsh}^{k+}, we assume that $x_{\delta(s)h}^{k}$ is 0 because when $x_{\delta(s)h}^{k}=1$, M_{lsh}^{k+} is not relevant. Thus, there is some $k'\neq k$ with $x_{\delta(s)h}^{k'}=1$, and $x_{\delta(s)h}^{k''}=0$, for $k''\in\mathcal{K}_{\delta(s)h}\setminus\{k,k'\}$. Thus, by (18) and (19), M_{lsh}^{k+} should have a value such that

$$M_{lsh}^{k+}\geq (q_{\delta(s)h})^{k'}\rho_{ls,h-1}-(q_{\delta(s)h})^{k}\rho_{ls,h-1}=((q_{\delta(s)h})^{k'}-(q_{\delta(s)h})^{k})\rho_{ls,h-1}. \tag{21}$$

Therefore, a reasonable value of M_{lsh}^{k+} can be obtained by deriving an upper bound of the right-hand-side of (21). By the definition of q_{jh}, we have $(q_{\delta(s)h})^{k'}-(q_{\delta(s)h})^{k}\leq 1-(q_{\delta(s)h})^{k}$. An upper bound of $\rho_{ls,h-1}$ can be derived as follows. By the definitions of Q_j and R_j and the assumption that each subsystem j must be configured with at least one component, we have $Q_j\leq \max_{h\in\mathcal{H}_j}\{q_{jh}\}$ and $R_j\leq 1$. This, in conjunction with the relationship between ρ_{lsh} and φ_{ls}, results in

$$\rho_{ls,h-1}\leq \varphi_{l,s-1}=\prod_{s'=1}^{s-1}O_{s'}\leq \prod_{s'=1,\delta(s')\in\mathcal{J}_l^Q}^{s-1}Q_{\delta(s')}\leq \prod_{s'=1,\delta(s')\in\mathcal{J}_l^Q}^{s-1}\Big(\max_{h'\in\mathcal{H}_{\delta(s')}}\{q_{\delta(s')h'}\}\Big). \tag{22}$$

Putting the above results together, we can derive a reasonable value for M_{lsh}^{k+}:

$$M_{lsh}^{k+}=(1-(q_{\delta(s)h})^{k})\prod_{s'=1,\delta(s')\in\mathcal{J}_l^Q}^{s-1}\Big(\max_{h'\in\mathcal{H}_{\delta(s')}}\{q_{\delta(s')h'}\}\Big).$$

Similarly, we can derive the value for $M_{l,s,h}^{k-}$:

$$M_{lsh}^{k-}=((q_{\delta(s)h})^{k}-(q_{\delta(s)h})^{K_{\delta(s)h}})\prod_{s'=1,\delta(s')\in\mathcal{J}_l^Q}^{s-1}\Big(\max_{h'\in\mathcal{H}_{\delta(s')}}\{q_{\delta(s')h'}\}\Big).$$

Note that inequality (22) also yields upper bounds for the decision variables ρ_{lsh} and φ_{ls}.

3.2. Tree-Based MILP Formulation

In Section 3.1, each product term in the reliability function in (5) is linearized separately without considering possible relationships with other product terms. However, different product terms in the reliability function may contain one or more common factors, suggesting their inherent interconnections. As an example, both product terms $Q_1Q_2Q_3$ and $Q_1Q_2R_3Q_4R_5$ contain factors Q_1 and Q_2. This observation motivates us to represent the product terms in the reliability function using a tree structure, which we refer to as a *reliability function tree* hereafter. For any given reliability function, such a tree is constructed such that each node in the tree, except the root node, represents a specific factor contained in a specific product term in the reliability function, and each product term in the reliability function is represented by a path from the root node to another node in the tree, where the paths for different product terms that share one or more common factors may share some nodes in the tree. Thus, such a tree representation establishes the underlying interconnections between different product terms in a reliability function, which are then used to reduce the number of decision variables and constraints needed during the linearization of these product terms.

For the reliability function in (5), where there are L product terms, the construction of a reliability function tree begins by establishing the root node and associating it with a label 1, followed by L steps, one for each product term. Step l considers the lth product term and performs the following: either finds a path from the root node to another node in the tree such that the labels associated with the nodes on this path are the same as the factors in this product term, respectively, or, if such a path does not exist, grows the tree by adding new nodes and arcs to either extend an existing path or add a new path from the root node such that the labels associated with the nodes on the extended path or the new path are the same as the factors in this product term, respectively.

In the following, we describe Step l of the tree construction in detail, for $l=1,\ldots,L$. This step considers the lth product term given in (5), which is $\prod_{j\in\mathcal{J}_l^Q}Q_j\prod_{j'\in\mathcal{J}_l^R}R_{j'}$. For ease of presentation, we borrow the same set of notations introduced in Section 3.1.1. Let $\mathcal{J}_l=\mathcal{J}_l^Q\cup\mathcal{J}_l^R$. As discussed in Section 3.1.1, $\mathcal{J}_l^Q$ and $\mathcal{J}_l^R$ do not overlap, and hence $|\mathcal{J}_l|=|\mathcal{J}_l^Q|+|\mathcal{J}_l^R|$. Let $\delta(s)\in\mathcal{J}$ denote the subsystem corresponding to the sth factor in the product term, that is, the sth factor in the lth product term is either $Q_{\delta(s)}$ or $R_{\delta(s)}$, for $s=1,\ldots,J_l$. Given the lth product term, Step l performs the following substeps:

Step l(1): Check if there is a path in the current tree from the root node to some other node such that (i) the path contains exactly J_l nodes in addition to the root node, and (ii) the label associated with the sth node on this path is equal to the sth factor in the product term, for $s=1,\ldots,J_l$. This can be done using a simple procedure as follows:

(1-a) Start at the root node by setting it as the current node, and set iteration counter $k = 1$.

(1-b) Check if the current node has any child node with the label equal to the kth factor in the product term. If the answer is yes, then traverse to this child node, set it as the current node, and let $k = k + 1$. Otherwise, stop.

(1-c) Repeat (1-b) until k reaches $J_l + 1$ or the procedure is stopped before k reaches $J_l + 1$.

If k reaches $J_l + 1$ in the above procedure, then a path is found that satisfies (i) and (ii), and go to Step l(2). Otherwise, no such path exists, and go to Step l(3).

Step l(2): In the case an existing path is found, mark the end node of this path by a star to indicate that the unique path from the root node to this starred node corresponds to the lth product term. The product of the labels associated with the nodes on this path yields the product term.

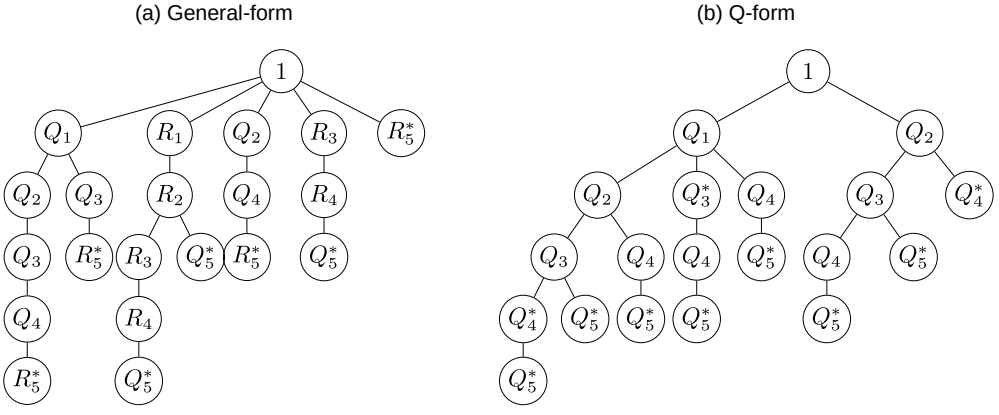
Step l(3): In the case no existing path is found, then the procedure in Step l(1) is stopped after visiting $k-1$ nodes besides the root node, where $k < J_l + 1$. Denote the last node visited as node ξ, which is the root node if $k = 1$. It can be seen that if $k > 1$, then the label associated with each of the $k-1$ nodes visited corresponds to each of the first $k-1$ factors in the lth product term, respectively. We grow the tree by creating $J_l - k + 1$ new nodes and associating the tth new node with a label equal to the $(k-1+t)$th factor in the lth product term, for $t = 1, \ldots, J_l - k + 1$, and creating an equal number of new arcs to connect node ξ and the first new node, and the tth and $(t+1)$st new nodes, for $t = 1, \ldots, J_l - k$, respectively. After this, the unique path from the root node to the last newly added node satisfies (i) and (ii) and hence represents the lth product term. Mark the last newly added node with a star.

Figure 2(a) shows the tree representation for the reliability function (11) constructed following the above procedures. Each node is associated with a label, which is 1 for the root node, and either Q_j or R_j with some $j \in \mathcal{J}$ for every other node. A node whose label has a superscript asterisk is a starred node. The unique path from the root node to each starred node in the tree corresponds to each specific product term in the reliability function. To illustrate, we briefly describe how the tree in Figure 2(a) is constructed. First, the root node with label 1 is created. Then, we execute seven steps, one for each product term in the reliability function (11). For the first product term in the reliability function, Step 1 has to create an entirely new path, $(1, Q_1, Q_2, Q_3, Q_4, R_5)$, which is the leftmost path shown in the tree. For the second product term, Step 2 first goes from the root node to the node labeled Q_1, then creates two new nodes with labels Q_3 and R_5, respectively, such that a new path $(1, Q_1, Q_3, R_5)$ is created, which is the second path from left shown in the tree. For the third product term, Step 3 needs to create an entirely new path $(1, R_1, R_2, R_3, R_4, Q_5)$, which is the third path from left in the tree. For the fourth product term, Step 4 traverses from the root node to the nodes labeled R_1 and R_2 and creates a new node labeled Q_5 to establish a new path $(1, R_1, R_2, Q_5)$. For each of the remaining three product terms, the corresponding step has to create an entirely new path, namely, $(1, Q_2, Q_4, R_5)$, $(1, R_3, R_4, Q_5)$ and $(1, R_5)$, respectively, because no existing path from the root node can be used as the starting segment of the path needed for the product term.

For ease of presentation, given a reliability function tree, we assign a distinct index to each node such that the root node is indexed as node 0, and all other nodes are indexed as node 1, node 2, ..., with the requirement that a child node has a larger index than its parent node. Let $\mathcal{N}$ be the set of all nodes excluding the root node, and $\mathcal{N}^S \subseteq \mathcal{N}$ the set of starred nodes in the tree. Let $\pi(n)$ denote the unique parent node of node $n \in \mathcal{N}$. Let $l(n)$ be the label of node $n \in \mathcal{N} \cup \{0\}$. Let $\mathcal{N}^Q \subseteq \mathcal{N}$ be the set of nodes with a label equal to certain Q_j and $\mathcal{N}^R \subseteq \mathcal{N}$ be the set of nodes with a label equal to certain R_j. Let $\eta(n)$ be the index of the subsystem corresponding to the label of node $n \in \mathcal{N}$, that is, $l(n) = Q_{\eta(n)}$ if $n \in \mathcal{N}^Q$ and $l(n) = R_{\eta(n)}$ if $n \in \mathcal{N}^R$.

To linearize all the product terms in a given reliability function, we utilize the corresponding reliability function tree as follows. We define a continuous decision variable for each node $n \in \mathcal{N}$, denoted as φ_n, to represent

Figure 2. Tree Representations of the Reliability Function of System 1



the *value* of this node, which is defined to be equal to the product of the labels of this node and all its predecessors (i.e., all the nodes on the path from the root node to this node). Define the value of the root node as a constant $\varphi_0 = 1$. Thus, the value of each starred node in the tree represents one specific product term. Accordingly, the reliability function can be stated as $f(x) = \alpha + \sum_{n\in\mathcal{N}^S}\beta_n\varphi_n$, where β_n is the coefficient of the product term that is represented by the value of node n. We describe below the relationships between the variables φ_n's, and how such relationships can be represented by linear inequalities and other variables.

By definition, the value of node n is the product of its label and the value of its parent node, that is, $\varphi_n = \varphi_{\pi(n)}l(n)$. Thus, we have

$$\varphi_n = \begin{cases} \varphi_{\pi(n)}Q_{\eta(n)}, & \forall n \in \mathcal{N}^Q, \\ \varphi_{\pi(n)}R_{\eta(n)} = \varphi_{\pi(n)} - \varphi_{\pi(n)}Q_{\eta(n)}, & \forall n \in \mathcal{N}^R. \end{cases} \tag{23}$$

It is necessary to linearize the nonlinear term $\varphi_{\pi(n)}Q_{\eta(n)}$ shared by the two cases in (23). By the definition of $Q_{\eta(n)}$ given by (3), we have

$$\varphi_{\pi(n)}Q_{\eta(n)} = \varphi_{\pi(n)}\prod_{h\in\mathcal{H}_{\eta(n)}}\left(\sum_{k\in\mathcal{K}_{\eta(n)h}}(q_{\eta(n)h})^k x^k_{\eta(n)h}\right). \tag{24}$$

Thus, we need to linearize the right-hand-side of (24). To this end, we introduce additional decision variables: $\rho_{n0} = \varphi_{\pi(n)}$ and ρ_{nh} is the product of $\varphi_{\pi(n)}$ and the first h factors in the product term involved in (24), that is, $\rho_{nh} = \varphi_{\pi(n)}\prod_{h'=1}^{h}(\sum_{k\in\mathcal{K}_{\eta(n)h'}}(q_{\eta(n)h'})^k x^k_{\eta(n)h'})$, for $n \in \mathcal{N}$ and $h \in \mathcal{H}_{\eta(n)}$. Hence, we have $\varphi_{\pi(n)}Q_{\eta(n)} = \rho_{nH_{\eta(n)}}$ and

$$\rho_{nh} = \rho_{n,h-1}\left(\sum_{k\in\mathcal{K}_{\eta(n)h}}(q_{\eta(n)h})^k x^k_{\eta(n)h}\right), \quad \forall n \in \mathcal{N}, h \in \mathcal{H}_{\eta(n)}. \tag{25}$$

Analogous to the analysis in Section 3.1, the nonlinear Equation (25) can be linearized as follows:

$$\rho_{nh} \le \rho_{n,h-1}(q_{\eta(n)h})^k + M^{k+}_{nh}(1 - x^k_{\eta(n)h}), \quad \forall n \in \mathcal{N},\ h \in \mathcal{H}_{\eta(n)}, k \in \mathcal{K}_{\eta(n)h}, \tag{26}$$

$$\rho_{nh} \ge \rho_{n,h-1}(q_{\eta(n)h})^k - M^{k-}_{nh}(1 - x^k_{\eta(n)h}), \quad \forall n \in \mathcal{N},\ h \in \mathcal{H}_{\eta(n)}, k \in \mathcal{K}_{\eta(n)h}, \tag{27}$$

where M^{k+}_{nh} and M^{k-}_{nh} are the big-M constants.

From the above analysis, the value of node $n \in \mathcal{N}$ is $\rho_{nH_{\eta(n)}}$. Consequently, the BINLP formulation (5)–(9) can be reformulated as the following MILP model:

$$\begin{aligned}
[\text{TREE}]\ \max\ & f(x) = \alpha + \sum_{n\in\mathcal{N}^S}\beta_n\varphi_n \\
\text{s.t.}\ & \varphi_0 = 1, \\
& \rho_{n0} = \varphi_{\pi(n)}, \quad \forall n \in \mathcal{N}, \\
& \rho_{nh} \le \rho_{n,h-1}(q_{\eta(n)h})^k + M^{k+}_{nh}(1 - x^k_{\eta(n)h}), \quad \forall n \in \mathcal{N},\ h \in \mathcal{H}_{\eta(n)}, k \in \mathcal{K}_{\eta(n)h}, \\
& \rho_{nh} \ge \rho_{n,h-1}(q_{\eta(n)h})^k - M^{k-}_{nh}(1 - x^k_{\eta(n)h}), \quad \forall n \in \mathcal{N},\ h \in \mathcal{H}_{\eta(n)}, k \in \mathcal{K}_{\eta(n)h}, \\
& \varphi_n = \rho_{nH_{\eta(n)}}, \quad \forall n \in \mathcal{N}^Q, \\
& \varphi_n = \varphi_{\pi(n)} - \rho_{nH_{\eta(n)}}, \quad \forall n \in \mathcal{N}^R, \\
& \sum_{j\in\mathcal{J}}\sum_{h\in\mathcal{H}_j}\sum_{k\in\mathcal{K}_{jh}} k a_{ijh}x^k_{jh} \le b_i, \quad \forall i \in \mathcal{I}, \\
& \sum_{k\in\mathcal{K}_{jh}} x^k_{jh} = 1. \quad \forall j \in \mathcal{J},\ h \in \mathcal{H}_j, \\
& \sum_{h\in\mathcal{H}_j}\sum_{k\in\mathcal{K}_{jh}\setminus\{0\}} x^k_{jh} \ge 1, \quad \forall j \in \mathcal{J}, \\
& 0 \le \rho_{nh} \le 1, \quad \forall n \in \mathcal{N},\ h \in \mathcal{H}_{\eta(n)}, \\
& x^k_{jh} \in \{0,1\}, \quad \forall j \in \mathcal{J},\ h \in \mathcal{H}_j,\ k \in \mathcal{K}_{jh}.
\end{aligned}$$

Following a similar approach as in Section 3.1, we can eliminate the φ_n variables. However, we choose to retain the φ_n variables and the corresponding constraints for the same reason given earlier. Moreover, the values of the

Big-M constants, M_{nh}^{k+} and M_{nh}^{k-}, can be determined using a similar approach as in Section 3.1.2. This is detailed in Online Appendix C.

3.3. Other Formulations and Enhancement

We discuss alternative MILP formulations that can be derived for the RAP. In Section 3.3.1, we convert the reliability function to a form where all product terms involve only Q_j's and discuss how the MILP formulations [BASIC] and [TREE] may change. In Section 3.3.2, we propose a different way to construct the reliability function tree and discuss how the formulation [TREE] changes based on this tree. Furthermore, in Online Appendix D, we discuss how the formulations would change under the alternative definition of binary variables presented in Section 2.1.

3.3.1. Q-Only Product Terms. The analysis in Section 3.1 is based on the *general-form* reliability function (1), where each product term may contain both Q_j and R_j factors. Now, we convert the reliability function such that all product terms only involve Q_j's for $j \in \mathcal{J}$. This can be done by replacing R_j with $1 - Q_j$ for all $j \in \mathcal{J}$ in (1). The resulting reliability function has the following form:

$$f(x) = \alpha + \sum_{l \in \mathcal{L}} \left(\beta_l \prod_{j \in \mathcal{J}_l^Q} Q_j \right). \tag{28}$$

To simplify the presentation, the function in (28) is called a Q-form function. For example, the Q-form of the reliability function (11) is

$$\begin{aligned} f_1(x) = 1 &- Q_1Q_3 - Q_2Q_4 + Q_1Q_2Q_3Q_4 - 2Q_1Q_2Q_3Q_4Q_5 - Q_2Q_3Q_5 + Q_1Q_2Q_3Q_5 \\ &- Q_1Q_4Q_5 + Q_1Q_2Q_4Q_5 + Q_1Q_3Q_4Q_5 + Q_2Q_3Q_4Q_5. \end{aligned} \tag{29}$$

As (28) is a special form of (1), the linearization procedures and formulations presented in Sections 3.1 and 3.2 remain valid. Accordingly, the MILP formulations [BASIC] and [TREE] can still be used for the RAP with a Q-form reliability function. We denote these MILP formulations for the RAP with a Q-form reliability function as [Q-BASIC] and [Q-TREE], respectively, to indicate the fact that the reliability function is converted to the Q-form in these formulations.

There are three potential advantages of using the Q-form reliability function (28) compared with the general-form (1). First, the formulations [Q-BASIC] and [Q-TREE] can be simpler to some extent than [BASIC] and [TREE], respectively. We compare [BASIC] and [Q-BASIC] in the following. A similar comparison between [TREE] and [Q-TREE] can be made and is thus omitted for brevity. In the original recursive equations in [BASIC], the value of φ_{ls} depends on whether the preceding factor is Q_j or R_j for some $j \in \mathcal{J}$. Thus, two inequalities, that is, (19) and (20), must be used to linearize the nonlinear expression, regardless of the sign of the product term's coefficient, β_l. However, for the Q-form reliability function, only Equation (19) is required for the case where $\beta_l > 0$ and only Equation (20) is required for the case where $\beta_l < 0$. The maximization nature of the problem ensures that with only a single inequality shown above, the decision variables take the correct values.

Second, for the Q-form reliability function, we can derive better bounds for the decision variables and smaller values for the big-M constants. This can be observed from Equation (22), where only the factors that are Q_j for some $j \in \mathcal{J}$ (i.e., these included in the set $\mathcal{J}_l^Q$) are considered in calculating the bounds. Thus, the formulations based on the general-form reliability function only consider a subset of factors, whereas the formulations based on the Q-form reliability function consider all factors, resulting in tighter variable bounds and smaller big-M values.

Third, in the Q-form reliability function, there are more common factors shared among different product terms. This feature can be beneficial for the reliability function tree. As an example, Figure 2(b) shows the reliability function tree for the Q-form reliability function $f_1(x)$ given in (29). Compared with the tree in Figure 2(a), which is for the general-form function (11), one can observe a reduction in node count by one and an increase in the number of starred nodes appearing as nonleaf nodes, suggesting that the paths representing different product terms exhibit more shared common factors. This advantage can be more significant when the tree is constructed following the specific scheme discussed in Section 3.3.2.

However, an obvious drawback of using the Q-form reliability function is that it may result in, but not always, more product terms. For example, the general-form reliability function (11) contains 7 product terms, whereas its Q-form (29) has 10 product terms. If the basic formulation is used, more product terms often result in a larger

number of continuous decision variables and constraints. If the tree-based formulation is used, the impact of having more product terms is less clear because it also depends on the reliability function tree.

3.3.2. Occurrence-Based Reliability Function Tree. Given a reliability function, the reliability function tree detailed in Section 3.2 is built by processing the product terms one at a time, following their sequence of appearance. The tree expands at each step based on a single product term only. To increase the sharing of nodes in the tree by multiple product terms, we propose an alternative tree construction method in this section. The key idea is to consider multiple product terms simultaneously, rather than processing them one-by-one, and to grow the tree at each step by expanding the nodes shared by multiple product terms. A key operation of this approach, which is repeated multiple times, involves finding the common factor that occurs most frequently in a set of product terms being considered and adding a node that represents this common factor. Thus, a tree constructed in this way is referred to as an *occurrence-based* reliability function tree.

Given a reliability function with L product terms with its form defined in (5), suppose that each product term contains at most V factors. We distinguish different parts of the tree to be constructed by levels, where the root node is defined as level 0, and level v consists of all child nodes of the nodes at level $v-1$, for $v=1,\ldots$. Since each product term contains at most V factors, the tree has V levels, excluding the root node level, because each factor of a product term is represented as a node at a different level of the tree. For this reliability function, the occurrence-based reliability function tree is constructed level-by-level. We first establish the root node, denoted as node 0, and associate it with a label 1 and a set of product terms, denoted as $\mathscr{T}(0)$, where $\mathscr{T}(0)=\mathscr{L}$. We then execute V steps, such that at the vth step, for $v=1,\ldots,V$, we build all the nodes needed at the vth level of the tree to represent certain common factors shared by some product terms, and associate each newly generated node with a label and a subset of the product terms. We describe Step v of the tree construction as follows. Step v checks every node at level $v-1$ of the tree (which is created in Step $v-1$). If $v=1$, the only node at level $v-1$ is the root node. For each node being checked, denoted as node n, generate one or more child nodes by the following procedures.

Step v(1): Let $\mathscr{U}(n)$ be the set of nodes on the path from the root node to node n, excluding the root node but including node n. In addition, we know the label and the set of product terms, $\mathscr{T}(n)$, associated with node n (which are created in Step $v-1$ or updated so far in Step v). The way the tree is constructed so far implies that every product term in $\mathscr{T}(n)$ contains all the factors represented by the nodes in $\mathscr{U}(n)$.

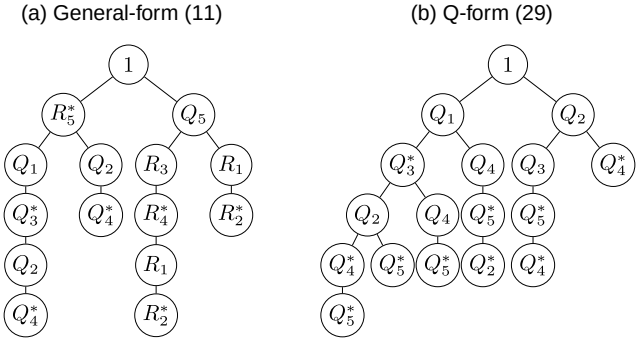
Step v(2): Find the most frequently occurring factor, excluding the factors represented by the nodes in $\mathscr{U}(n)$, in all the product terms in $\mathscr{T}(n)$ with an arbitrary tie-breaking rule. For this factor, build a child node of node n, denoted as node n', to represent this factor. Add an arc to connect node n and node n', and associate node n' with (i) a label equal to this factor, and (ii) a set of product terms, denoted as $\mathscr{T}(n')$, which is the subset of $\mathscr{T}(n)$ containing all the product terms that contain this factor.

Step v(3): If $\mathscr{T}(n')$ contains all the product terms in $\mathscr{T}(n)$ (i.e., $\mathscr{T}(n')=\mathscr{T}(n)$), then check the next node at level $v-1$. If all the nodes at level $v-1$ have been checked, then Step v is completed and stop. Otherwise, go to the next node at level $v-1$, denote it as node n, and check this node by going back to Step v(1) to repeat the above procedures.

Step v(4): If $\mathscr{T}(n')$ does not contain all the product terms in $\mathscr{T}(n)$, update the set of product terms associated with node n, $\mathscr{T}(n)$, as $\mathscr{T}(n)\setminus\mathscr{T}(n')$, and check node n again by going back to Step v(1) to repeat the above procedures.

Following the above procedure, we can generate two occurrence-based trees shown in Figure 3 for System 1, where (a) is the tree for the general-form reliability function (11), and (b) is the tree for the Q-form reliability

Figure 3. Occurrence-based Tree Representations of the Reliability Function of System 1



function (29). Compared with the original trees presented in Figure 2, in the occurrence-based trees, the node count decreases from 20 to 14 for the general-form reliability function (11), and from 21 to 16 for the Q-form reliability function (29). We note that although the original tree for the Q-form reliability function has fewer nodes than that for the general-form reliability function, their corresponding occurrence-based trees exhibit the opposite result.

In general, for any reliability function with a general-form given in (5), the above procedure can be used to construct an occurrence-based tree. Given such a tree, the tree-based MILP formulation [TREE] given in Section 3.2 can be applied. We denote the resulting formulation as [OB-TREE]. Similarly, for any reliability function with a Q-form given in (28), the above procedure can also be applied to generate an occurrence-based tree. Applying the formulation [TREE] to this occurrence-based tree, we denote the resulting formulation as [Q-OB-TREE].

3.3.3. Enhancement with Valid Inequalities. In the following, we derive two valid inequalities for the [TREE] model developed in Section 3.2. A similar procedure can be applied to derive similar valid inequalities for other formulations, as detailed in Online Appendix E. The first valid inequality is directly derived from the nonlinear Equation (25). Let $\overline{\rho}_{nh}$ denote the upper bound of ρ_{nh}, which is derived in Online Appendix C. Then, combining the fact that $\rho_{n,h-1} \le \overline{\rho}_{n,h-1}$ and (25) yields

$$\rho_{nh} \le \overline{\rho}_{n,h-1} \sum_{k \in \mathscr{K}_{\eta(n)h}} (q_{\eta(n)h})^k x^k_{\eta(n)h}, \quad \forall n \in \mathcal{N}, h \in \mathscr{H}_{\eta(n)}. \tag{30}$$

We now construct the second valid inequality. Let $\mathscr{G}_n$ represent the set of node labels on the path from the root to node n in the reliability function tree. By the definition given in Section 3.2, the value of node n is the product of the labels contained in $\mathscr{G}_n$. Thus, for any pair of nodes, denoted as n and n', such that $\mathscr{G}_n \supset \mathscr{G}_{n'}$, their respective values must have the following relationship:

$$\rho_{nH_{\eta(n)}} \le \rho_{n'H_{\eta(n')}}, \ \forall n, n' \in \mathcal{N}. \tag{31}$$

As an example, we consider two paths, $Q_1Q_2Q_3Q_4R_5^*$ and $Q_2Q_4R_5^*$, in the reliability function tree shown in Figure 2(a). Because these paths do not overlap, the interconnections between their corresponding product terms are not captured by the tree representation. However, since the label set of the second path is encompassed within the label set of the first path, we can add a valid inequality of the form (31). In general, for any pair of nodes, n and n', such that $\mathscr{G}_n \supset \mathscr{G}_{n'}$, if the path from the root to node n' is not entirely overlapped with the path from the root to node n (i.e., the latter does not contain the former completely), then a valid inequality of form (31) can be added to the [TREE] model. On the other hand, if the path from the root to node n contains the entire path from the root to node n', then (31) is already implied in the constraints of the [TREE] model.

3.4. Comparison of the Models

In the following, we compare the various MILP formulations derived in the previous subsections. We first observe that all these models utilize the same set of binary variables as in the original BINLP formulation (5)–(9) given in Section 2.1, where the number of binary variables is $\sum_{j \in \mathscr{J}} \sum_{h \in \mathscr{H}_j} K_{jh}$ by definition. This implies that the linearization techniques used in these models do not alter the number of binary variables. Conversely, the number of continuous variables and the number of constraints in a formulation depend on the specific model used and the specific data in the instance, as discussed below.

In model [BASIC], the numbers of continuous variables, ρ_{lsh} and φ_{ls}, are $\sum_{l=1}^{|\mathscr{L}|} \sum_{s=1}^{J_l} H_{\delta(s)}$ and $\sum_{l=1}^{|\mathscr{L}|} J_l$, respectively, and the number constraints is

$$|\mathscr{L}| + 2 \times \sum_{l=1}^{|\mathscr{L}|} J_l + 2 \times \sum_{l=1}^{|\mathscr{L}|} \sum_{s=1}^{J_l} \sum_{h=1}^{H_{\delta(s)}} K_{\delta(s)h} + |\mathscr{I}| + \sum_{j=1}^{|\mathscr{J}|} H_j + |\mathscr{J}|.$$

As for model [TREE], the numbers of continuous variables, ρ_{nh} and φ_n, are $\sum_{n=1}^{|\mathcal{N}|} H_{\eta(n)}$ and $|\mathcal{N}|$, respectively, and the number of constraints is

$$1 + 2 \times |\mathcal{N}| + 2 \times \sum_{n=1}^{|\mathcal{N}|} \sum_{h=1}^{H_{\eta(n)}} K_{\eta(n)h} + |\mathscr{I}| + \sum_{j=1}^{|\mathscr{J}|} H_j + |\mathscr{J}|.$$

Thus, the numbers of continuous variables and constraints in [BASIC] are determined by the number of product terms in the reliability function, $|\mathscr{L}|$, the number of factors in each of these terms, J_l, and the number of component types in each subsystem H_j, among others, whereas the size of [TREE] depends on the number of nodes in

the reliability function tree, $|\mathcal{N}|$, and the number of component types in each subsystem, among others. Based on the construction of the reliability function tree, it is evident that a tree-based formulation ([TREE] or [Q-TREE]) always contains fewer or an equal number of continuous variables and constraints compared with the basic formulation ([BASIC] or [Q-BASIC], respectively).

Since there are usually more product terms in the Q-form reliability function than in the general-form reliability function, the basic formulation based on the Q-form reliability function, [Q-BASIC], usually contains more continuous variables and constraints than its counterpart [BASIC], which is based on the general-form reliability function. However, the impact of the reliability function form on the number of continuous variables and the number of constraints in a tree-based formulation is unclear, that is, no definitive conclusions can be drawn regarding the size difference between [TREE] and [Q-TREE], and between [OB-TREE] and [Q-OB-TREE]. There are other factors that influence the size difference between these formulations.

Finally, since the occurrence-based tree representation explores the common factors shared by different product terms, the formulations based on this tree representation, that is, [OB-TREE] and [Q-OB-TREE], usually have fewer nodes and hence fewer continuous variables and constraints than those based on the original tree representation, that is, [TREE] and [Q-TREE], respectively.

More detailed comparisons of these formulations are made in Section 4 based on specific instances of the various complex systems tested there.

4. Computational Experiments

In this section, we conduct computational experiments to compare the performance of the proposed MILP models, including the two foundational models, [BASIC] and [TREE], and their variants, [Q-BASIC], [Q-TREE], [OB-TREE], and [Q-OB-TREE]. All these models are solved using the Gurobi solver with default parameters.

In addition, we compare our method against a benchmark approach. The benchmark method is an extension of the branch-and-bound algorithm proposed by Ha and Kuo (2006), which was originally designed to solve the RAP for systems with homogeneous components. Since Ha and Kuo's algorithm cannot be directly applied to the systems with heterogeneous components studied in this paper, we extend it to make it applicable to such systems. A brief description of this procedure is provided in Online Appendix F. To the best of our knowledge, there is no existing branch-and-bound-based exact algorithm for systems with heterogeneous components. Thus, we make a contribution by designing the first such algorithm for such systems.

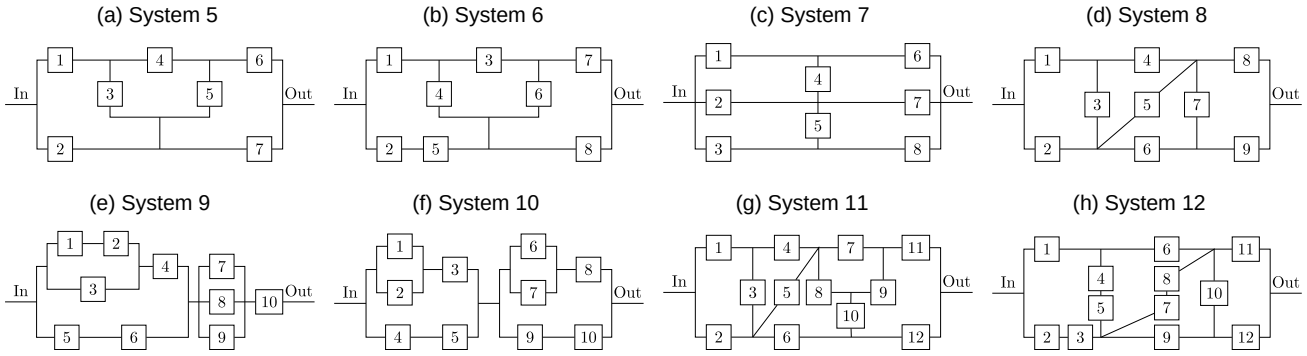
We use three sets of test instances with varying sizes and complexity: (i) the test instances used by an existing paper that studies the same problem as ours, and (ii) two new sets of test instances derived based on several complex systems considered in the literature. A detailed description of the test instances is given in Section 4.1. All the experiments are implemented using C++ and the Gurobi 11.0 solver on a workstation with a Xeon® W9-3495X processor and 512 GB of memory. Source codes, test instances, and detailed computational results are available at the GitHub software repository associated with this paper (Feng et al. 2025).

4.1. Test Instances

While there are well-known benchmark instances for the RAP for series-parallel systems, such as those proposed by Fyffe et al. (1968) and Coit and Konak (2006), to our best knowledge, the most relevant existing instances for heterogeneous complex systems are those created by Park (2020), which have a total of 48 instances. Thus, these instances are used in our experiments. These instances are generated based on the four complex systems, Systems 1, 2, 3, and 4, given in Figure 1, for which the reliability functions are given by (11)–(14), respectively. The number of subsystems in these systems varies from 5 to 7. We describe briefly how these instances are generated in Park (2020). For each system, three values for the number of component types are considered, $H_j \in \{2,3,4\}$, where H_j is identical for all $j \in \mathcal{J}$. Each component consumes two types of resources, that is, $I = 2$. The other parameters are set as $r_{jh} = \lfloor (0.5 + 0.05\sqrt{H_j} + 0.2\gamma_{jh}) \times 100 \rfloor / 100$, $a_{ijh} = \lfloor (1 + \gamma + 3\gamma_{jh}) \times 100 \rfloor / 100$, $b_i = \lfloor \sum_{j \in \mathcal{J}} \min_{h \in \mathcal{H}_j} a_{ijh} (1.5 + 0.5\gamma) \rfloor$, where γ_{jh} and γ are random numbers between 0 and 1. The way b_i's are generated ensures enough resources for configuring at least one component at each subsystem. For each value of H_j, four instances are randomly generated. Thus, 12 instances are generated based on each system, yielding a total of 48 instances. The data of the instances and their best-found solutions are available in the online supplement of Park (2020). No proven optimal solutions for these instances have been reported in the literature. We refer the interested reader to Park (2020) for further details.

To further evaluate the proposed models using larger complex systems than those in Figure 1, we generate two new sets of instances as follows. These instances are generated based on the eight complex systems (numbered as System 5, … , System 12) depicted in Figure 4, where the number of subsystems ranges from 7 to 12.

Figure 4. Complex Systems for Random Instances



These complex system structures are considered in many existing studies. Specifically, the structures of Systems 5, 8 and 11 are considered by Yeh (2017), Xu and Pi (2019), Yeh et al. (2010), and Yeh et al. (2022), the structures of Systems 6, 7, and 12 are considered by Yeh et al. (2010) and Xu and Pi (2019), and the structures of Systems 9 and 10 are considered by Ha and Kuo (2006). Both Yeh (2017) and Xu and Pi (2019) study the RAP for complex systems with heterogeneous components, but their focus is on designing heuristic methods. In addition, the objective of Xu and Pi (2019) is to minimize the cost such that a given level of reliability is achieved. Yeh et al. (2010) aim to estimate the reliability of a given system structure. Yeh et al. (2022) study a different RAP that also involves decisions on the reliability of components. Ha and Kuo (2006), as reviewed earlier, consider the RAP for complex systems with homogeneous components under linear or nonlinear constraints.

The reliability functions for these systems are provided in Online Appendix G. To be consistent with the way the first set of instances is generated, we generate test instances based on Systems 5–12 following the same procedure used by Park (2020), that is, the procedure described above for the systems in Figure 1. Thus, for each of these systems, we consider $H_j \in \{2,3,4\}$, and for each case of H_j, generate four instances randomly, resulting in 12 instances for each system and 96 instances in total. We note that the scale of the generated instances is competitive with those in Yeh (2017). We also note that, as discussed above, Yeh (2017) and Xu and Pi (2019) study the same problem as ours but focus on heuristics. The instances we generate for the three systems they also consider (i.e., Systems 5, 8 and 11) are slightly different from theirs, but the sizes of our instances are comparable to the sizes of theirs. Their instances are generated by adapting existing benchmarks for the RAP involving series-parallel systems. Each subsystem comprises at most three or four types of components, each consuming two types of resources.

We note that in general, the complexity of a test instance generated based on a given system depends on both the number of subsystems in the given system and the structure of the given system. It can be seen that Systems 5–12 are larger and more complex than Systems 1–4. Thus, the reliability functions for Systems 5–12 (see Online Appendix G) generally contain more product terms than those for Systems 1–4 (see (11)–(14)). Consequently, the second and third sets of test instances, which are generated based on Systems 5–12, are larger in scale, more complex, and hence harder to solve than the first set of instances, which are generated based on Systems 1–4.

Table 1 summarizes some key statistics of the reliability functions for the 12 complex systems. In particular, for both the general-form and the Q-form reliability functions of each system, we present the number of product terms in the reliability function (#Term), and the average number of factors in each of these terms (Avg. Length). Furthermore, for both the original tree representation and the occurrence-based tree representation, we present the number of nodes in the tree (#Node), the number of branches (equal to the number of leaf nodes, #Branch) in the tree, and the average number of nodes in each of these branches (Avg. Length). Since the general-form reliability function of a system may change when R_j is replaced with $(1-Q_j)$ or Q_j is replaced with $(1-R_j)$, for certain $j \in \mathcal{J}$, the values reported in Table 1 in the columns for the general-form reliability function may change accordingly. From Table 1, we can first see that the 12 systems can be divided into three groups in terms of the relative sizes of their reliability functions and representation trees: Systems 1–4 have small sizes, Systems 5–10 have medium sizes, and Systems 11–12 have large sizes. Furthermore, we can make the following observation for each given system: (i) the Q-form reliability function typically has a larger number of product terms than the same function in the general form; (ii) the occurrence-based reliability function tree almost always has fewer nodes than the original reliability function tree; (iii) the number (resp. average length) of branches in both tree presentations is less than the number (resp. average length) of terms in the original reliability function; and (iv) the average length of a branch in the occurrence-based reliability tree is less than that in the original reliability tree.

Table 1. Key Statistics of the Instances

						Tree			Occurrence-based tree		
Set	System	J	Function	#Term	Avg. length	#Node	#Branch	Avg. Length	#Node	#Branch	Avg. Length
1	1	5	General-form	7	3.29	21	7	3.29	15	4	4.00
			Q-form	20	3.50	20	8	3.63	17	6	3.83
	2	5	General-form	5	3.00	14	5	3.00	11	3	3.67
			Q-form	21	3.56	21	9	3.56	14	5	3.80
	3	6	General-form	9	3.67	29	9	3.67	19	5	4.40
			Q-form	23	4.20	23	8	4.25	19	6	4.50
	4	7	General-form	13	4.08	44	13	4.08	29	10	4.60
			Q-form	48	4.50	48	18	4.67	43	13	5.15
2	5	7	General-form	19	4.42	53	12	5.17	38	10	5.30
			Q-form	51	4.77	51	19	5.00	38	14	5.21
	6	8	General-form	24	5.00	64	15	5.73	49	12	6.08
			Q-form	117	5.20	117	43	5.56	86	34	5.62
	7	8	General-form	25	5.20	87	18	5.83	56	13	6.23
			Q-form	77	5.71	77	24	5.92	57	19	6.16
	8	9	General-form	48	5.92	132	31	6.61	99	25	6.72
			Q-form	128	6.06	128	45	6.44	108	37	6.54
	9	10	General-form	14	5.64	51	14	5.64	35	7	7.14
			Q-form	142	5.79	142	44	6.23	136	44	6.23
	10	10	General-form	9	6.67	29	6	6.83	26	5	7.20
			Q-form	133	5.45	133	60	5.87	108	50	5.90
3	11	12	General-form	178	8.13	618	135	8.47	369	91	9.04
			Q-form	789	7.86	789	327	8.08	626	253	8.35
	12	12	General-form	103	7.70	264	71	8.24	192	57	8.58
			Q-form	1,734	7.48	1,734	653	7.88	1,240	528	8.09

Consequently, we can make the following general conclusions, which are consistent with some of the observations made in Section 3.4, and with the test results shown in Section 4.2. For the same system, the MILP formulation [Q-BASIC] for an instance of the Q-form reliability function typically has a larger number of continuous variables and constraints than the formulation [BASIC] for the general-form reliability function. Similarly, the MILP models for an instance formulated from an occurrence-based reliability function tree, that is, [OB-TREE] and [Q-OB-TREE], usually have fewer continuous variables and constraints than those formulated from an original tree representation, that is, [TREE] and [Q-TREE], respectively. Finally, the MILP formulations for the instances generated for Systems 11 and 12 are expected to be much larger than those for Systems 5–10, which are expected to be larger than those for Systems 1–4. Thus, the 12 systems used in our computational experiment represent a wide range of problem sizes and difficulty levels. We note that the actual size of a test instance for a system depends on both the size of the reliability function or its tree representation, and the values of H_j. In the subsequent section, where the computational results are reported, the specific size of each instance under each specific MILP formulation is reported.

4.2. Computational Results

Results for the first set of test instances, which are from Park (2020), corresponding to the four systems in Figure 1, are presented in Section 4.2.1. Results for the second and third sets of test instances, which we generate corresponding to the eight systems in Figure 4, are reported in Sections 4.2.2 and 4.2.3, respectively. Some general insights from these three sets of tests are summarized in Section 4.2.4. Finally, our method is compared with the benchmark extended from the branch-and-bound algorithm of Ha and Kuo (2006) in Section 4.2.5. The detailed computational results are presented in Online Appendix H.

4.2.1. Result for the First Set of Instances. For each of the 48 instances, the size of each MILP model is presented in Table 2 in Online Appendix H, where column (S, J, H) indicates that the instance is generated from System S with the number of subsystems J and $H_j = H$, column "No." is the index of the instance, and columns #Vars, #Bins and #Cons represent the total number of variables (including binary and continuous), the number of binary variables, and the number of constraints, respectively. As all these models have exactly the same number of binary variables for the same instance, this number is only presented under the [BASIC] model.

Table 2 also shows the computational time in seconds (Time) taken to find an optimal solution of the six MILP models for each of the 48 instances. The six models are all extremely effective for all instances with respect to the

computational time performance. Each MILP model solves all instances to optimality very quickly, with the time taken no more than 24 seconds in the worst case. Most instances are solved to optimality by all models within one second.

We also observe that, for the same system, a larger number of component types H typically yields more variables and constraints in any of the models, which typically renders the problem more difficult to solve. Comparing these models based on the average computational times for each subset of four instances for a given (S, J, H), we can observe that the [Q-OB-TREE] model performs the best overall, although it does not dominate any of the other models over all subsets of instances, and the computational time differences between [Q-OB-TREE] and the other models are not significant.

As mentioned earlier, Park (2020) develops a sample-based MILP model and an aggregation-based algorithm that solves the MILP model with a given tolerance. Only when the sample size approaches infinity, the solution from their MILP model algorithms converges to an optimal solution, and when the tolerance approaches 0, the solution from their aggregation-based algorithm converges to the optimal solution of the MILP model. A direct comparison between our results and theirs is not feasible because (i) the solutions generated by our models are guaranteed to be optimal, and (ii) our computational platform is different from theirs. Nonetheless, we summarize their computational results on the instances with $H = 4$. In their experiments, they use four sample sizes in $\{1000, 2000, 4000, 8000\}$. The average computational time taken by their sample-based MILP model for each sample size ranges from 2.3 to 122.4 seconds, 2.9 to 152.5 seconds, 3.2 to 149.4 seconds, and 6.8 to 403.8 seconds, respectively. Their aggregation-based heuristic typically requires more time than their MILP model. For example, with a tolerance of 10^{-5}, the heuristic takes an average computational time ranging from 9.9 to 259.4 seconds, 10.9 to 287.7 seconds, 9.6 to 282.8 seconds, 16.6 to 653.3 seconds, respectively, for the four sample sizes. They conjecture that their best solutions are very likely to be optimal, as all their algorithms converge to a specific value as the sample size increases for each instance. Our computational results confirm that their conjecture is true.

4.2.2. Result for the Second Set of Instances. The computational results for the second set of instances, that is, those generated based on Systems 5 to 10, are given in Tables 3 and 4 in Online Appendix H. These tables are organized in the same way as Table 2. Comparing the results in these tables with those in Table 2, we observe that these instances yield larger models (i.e., more variables and constraints) than the first set of instances. As a result, these instances are generally more challenging to solve than the first set, as evidenced by the increased computational time. Now, comparing the different models, we can make the following observations. First, all the models are very effective. Every model solves every instance within 2,000 seconds and solves most instances within 600 seconds. Second, the tree-based models contain significantly fewer variables and constraints than the basic models. Consequently, the [TREE] and [OB-TREE] models take less computational time than the [BASIC] model in most cases. However, the [Q-TREE] and [Q-OB-TREE] models require similar computation time as the [Q-BASIC] model, despite the differences in model size. Third, the three models based on the Q-form reliability function outperform those based on the general-form reliability function on the instances associated with Systems 5–8, whereas the opposite holds on the instances associated with Systems 9 and 10. This is mainly due to the structures of these systems. A close examination of Table 1 shows that the general-form reliability functions for Systems 9 and 10 are quite compact, whereas their Q-form reliability functions contain a much larger number of product terms, and consequently, the corresponding models contain significantly more variables and constraints. Finally, although none of the models outperforms all the others over most instances, the [Q-OB-TREE] model outperforms the other models over more than half of the instances, and hence can be viewed as the best approach overall.

4.2.3. Result for the Third Set of Instances. Now, we discuss the results based on the third set of instances, that is, those derived from Systems 11 and 12. As these instances are much more challenging, we set a 24-hour limit on the computational time for each instance. The computational results are presented in Table 5 in Online Appendix H, which follows a similar structure to that in Table 2. For any instance not solved within the time limit by a specific method, the corresponding computational time is entered as "T.L.". In addition, we provide the MIP gap (%), which is the relative gap between the best feasible solution and the best lower bound found within the time limit, for any specific model and instance when this instance is not solved by this model within the time limit. Consequently, the average computational time for the four instances per parameter combination (S, J, H) is the mean of the computational times for the instances solved to optimality within the time limit. Similarly, the average MIP gap is the mean of the MIP gaps for the instances not solved to optimality within the time limit.

The model sizes for these instances, in terms of the number of variables and constraints, are substantially larger than those for the first and second sets of instances. Among the six models, only [Q-TREE] and [Q-OB-TREE] can solve all instances to optimality within the time limit. A careful examination shows that [Q-OB-TREE] outperforms [Q-TREE] as the former requires much less time than the latter for most instances. Furthermore, for the instances that can be optimally solved by all six models, the [Q-OB-TREE] model always shows better or comparable performance compared with the other models. This again demonstrates that the [Q-OB-TREE] model performs the best overall. We can also observe that both [BASIC] and [Q-BASIC] are unable to solve 9 of the 24 instances to optimality within the time limit, whereas [TREE] and [OB-TREE] fail to solve 4 and 2 instances to optimality within the time limit, respectively. This shows that the tree-based models outperform their basic versions for these challenging instances.

4.2.4. Some Insights. From the results reported in the previous subsections, we can observe that the performance of each model is influenced by many factors. First of all, the size of the formulation, represented by the number of variables and constraints, serves as a strong indicator of the underlying problem instance's difficulty. Thus, factors that influence the formulation size of a given instance have a significant impact on the computational effort required to solve the instance. For all the models, when the number of component types in an instance increases, the formulation size also increases. Consequently, we observe that every model almost always takes more time to solve the instances of a given system as the number of component types rises from $H = 2$ to $H = 4$. We also observe that for each instance, the [OB-TREE] model generally has fewer variables and constraints than the [TREE] model, which in turn generally has fewer variables and constraints than the [BASIC] model. As a result, the computational times required by models [BASIC], [TREE] and [OB-TREE] for a given instance typically exhibit a decreasing trend.

Besides the formulation size, the tightness of the linear relaxation of a formulation also has an impact on the required computational effort. According to the analysis in Section 3.3, the models based on the Q-form reliability function, namely [Q-BASIC], [Q-TREE], and [Q-OB-TREE], usually lead to tighter linear relaxations than those based on the general-form reliability function. This explains the fact that, in many instances, [Q-BASIC] has a much larger size but takes less time than [BASIC]. A similar observation can also be made with respect to the relationship between [Q-TREE] and [TREE] and between [Q-OB-TREE] and [OB-TREE]. Finally, there are also factors independent of the model formulations themselves that influence the computational time. It is known that MIP solvers are subject to unexpected variability (Lodi and Tramontani 2013). For example, by varying the value of a random seed used in the solver, we may see a large deviation in the computational time on the same instance.

4.2.5. Comparison with Benchmark Method. We conduct two comparative tests to compare our model [Q-OB-TREE], which, as shown above, performs the best overall among our six models, with a modified version of the branch-and-bound algorithm of Ha and Kuo (2006). Online Appendix F provides a brief description of the modified branch-and-bound algorithm, which we denote as "BB" for ease of reference. In the first test, we compare the computational times of the benchmark approach BB and our model [Q-OB-TREE] for solving the three sets of instances used in Sections 4.2.1–4.2.3. In the second test, we evaluate the robustness and scalability of these approaches by applying them to solve some variants of the above instances.

In the first test, we set a one-hour time limit for each instance associated with Systems 1–10, and a 48-hour time limit for each instance associated with Systems 11–12. The computational results are summarized in Table 6 in Online Appendix H. For the first set of instances (those associated with Systems 1–4), the benchmark approach BB is faster than our [Q-OB-TREE] model for every instance, although both are extremely efficient. For the second set of instances (those associated with Systems 5–10), neither method dominates the other. While BB performs better than [Q-OB-TREE] on most instances with $H = 2$ component types, [Q-OB-TREE] becomes more competitive as H increases to 3 and 4. For the third set of instances, we first note that the BB method fails to solve two instances within the time limit, whereas [Q-OB-TREE] solves every instance within the time limit. For the instances solved by both methods within the time limit, neither method dominates the other. Taking into account all the results in Table 6, we observe that for the relatively easy instances (those solvable within ten seconds), the BB method typically requires less time than the [Q-OB-TREE] model. However, for the more challenging instances, [Q-OB-TREE] outperforms the BB method, with a substantial advantage for many instances.

For the second test, which is to evaluate the robustness and scalability of BB and [Q-OB-TREE], we generate two new groups of instances based on the first two sets of instances used earlier (i.e., those associated with Systems 1–10) by changing two key parameters, H and γ_{jh}. In the first group of instances, the number of component

types H in a subsystem is 5 or 6. Thus, this group of instances has more component types than in the instances used before (which have 2, 3 or 4 component types). In the second group of instances, the number of component types H varies from 2 to 6, and the parameter γ_{jh}, which controls the reliability value of a component, is drawn randomly from [0, 0.5]. Recall that in the instances used earlier, $\gamma_{jh} \in [0,1]$. Thus, there is less variability in the reliability values of the components in the second group of new test instances than in the instances used earlier. We use such test instances for the following reasons: (i) problems with a larger number of components H are more difficult to solve, which motivates us to try larger values of H than in the earlier instances, (ii) problems with a smaller variability of reliability values are more difficult to solve, which motivates us to try a narrower interval [0, 0.5] for γ_{jh}, and (iii) instances associated with systems 11 and 12 with a larger H or/and a narrower interval for γ_{jh} may not be solvable by either of the methods, and hence they are not used.

For each instance in the first group, we set a one-hour time limit. The computational results for the first group of instances are reported in Table 7 in Online Appendix H. The BB method fails to solve 18 instances within the one-hour time limit, whereas the [Q-OB-TREE] model solves every instance to optimality in no more than 1,800 seconds. Furthermore, for instances solved by both methods within the time limit, the [Q-OB-TREE] model typically requires less computational time than the BB method. These results show that the [Q-OB-TREE] model is more robust and scalable than the BB method as the number of component types increases.

We now discuss the results for the second group of instances. The [Q-OB-TREE] model can solve all but three instances to optimality within one hour. Thus, for all instances except these three, we set a one-hour time limit for both methods. For each of these three instances, the time limit for BB is set to be the time taken by the [Q-OB-TREE] model to solve the instance to optimality. The computational results for the second group of instances are reported in Table 8 in Online Appendix H. The BB method fails to solve several instances of Systems 4 and 5, as well as most instances of Systems 6–10 with 4 or more component types, within the time limit. Thus, we can conclude that our [Q-OB-TREE] model substantially outperforms the BB method overall. In addition, comparing the results in Tables 7 and 8, we can see that for the instances of Systems 1–5 with $\gamma_{jh} \in [0,0.5]$, the computational times taken by the [Q-OB-TREE] model are similar to those for the instances with $\gamma_{jh} \in [0,1]$, whereas the computational times taken by BB for the instances with $\gamma_{jh} \in [0,0.5]$ are significantly higher than the results for the instances with $\gamma_{jh} \in [0,1]$. These results show that the [Q-OB-TREE] model is more robust and scalable than the BB method as the number of component types in the instance increases and/or the variability of the reliability values of components in the instance decreases.

5. Conclusion

In this paper, several exact MILP models are proposed for the RAP in heterogeneous complex systems. Our MILP models are the first such models that are formulated by linearizing the nonconvex nonlinear reliability function for a complex system. The MILP models are formulated based on both the general-form and Q-form reliability functions as well as on the two tree representations of these functions. The computational experiments have demonstrated the effectiveness of the proposed models on a diverse set of test instances. To the best of our knowledge, this is the first time all benchmark instances for this problem available in the literature have been solved to optimality within a short amount of time. In addition, we have also shown that our models are effective for larger instances generated based on more complex systems that have been considered in the literature.

Our experiments have shown that, among the six MILP formulations, the formulation built based on the Q-form reliability function and its occurrence-based tree representation achieves the best overall performance. Several future research directions are possible. First, our proposed formulation techniques can be applied to other versions of redundancy allocation problems, such as those discussed in Section 1. Second, it will be interesting to extend the linearization techniques to more complex RAPs involving, for example, both active and cold standby redundancy, and switching failure (Coit 2003, Kong et al. 2015). Finally, one could also devise similar techniques for other nonconvex optimization problems.

Acknowledgments

The authors are grateful for the insightful feedback provided by the reviewers and editors.

References

Bellman R, Dreyfus S (1958) Dynamic programming and the reliability of multicomponent devices. *Oper. Res.* 6(2):200–206.
Bulfin RL, Liu CY (1985) Optimal allocation of redundant components for large systems. *IEEE Trans. Reliability* 34(3):241–247.
Caserta M, Voß S (2015a) A discrete-binary transformation of the reliability redundancy allocation problem. *Math. Probl. Engrg.* 2015(1):276234.
Caserta M, Voß S (2015b) An exact algorithm for the reliability redundancy allocation problem. *Eur. J. Oper. Res.* 244(1):110–116.
Chang KH, Kuo PY (2018) An efficient simulation optimization method for the generalized redundancy allocation problem. *Eur. J. Oper. Res.* 265(3):1094–1101.

Chern MS (1992) On the computational complexity of reliability redundancy allocation in a series system. *Oper. Res. Lett.* 11(5):309–315.
Coit DW (2003) Maximization of system reliability with a choice of redundancy strategies. *IIE Trans.* 35(6):535–543.
Coit DW, Konak A (2006) Multiple weighted objectives heuristic for the redundancy allocation problem. *IEEE Trans. Reliability* 55(3):551–558.
Coit DW, Smith AE (1996) Solving the redundancy allocation problem using a combined neural network/genetic algorithm approach. *Comput. Oper. Res.* 23(6):515–526.
Coit DW, Smith AE (2002) Genetic algorithm to maximize a lower-bound for system time-to-failure with uncertain component Weibull parameters. *Comput. Indust. Engrg.* 41(4):423–440.
Coit DW, Zio E (2019) The evolution of system reliability optimization. *Reliability Engrg. System Safety* 192:106259.
Coit DW, Smith AE, Tate DM (1996) Adaptive penalty methods for genetic optimization of constrained combinatorial problems. *INFORMS J. Comput.* 8(2):173–182.
Devi S, Garg H, Garg D (2023) A review of redundancy allocation problem for two decades: Bibliometrics and future directions. *Artificial Intelligence Rev.* 56:7457–7548.
Djerdjour M, Rekab K (2001) A branch and bound algorithm for designing reliable systems at a minimum cost. *Appl. Math. Comput.* 118(2–3):247–259.
Feng J, Chen ZL, Che A, Chu C (2025) Exact MILP models for redundancy allocation with mixed components. https://doi.org/10.1287/ijoc.2024.0842.cd, https://github.com/INFORMSJoC/2024.0842.
Fyffe DE, Hines WW, Lee NK (1968) System reliability allocation and a computational algorithm. *IEEE Trans. Reliability* 17(2):64–69.
Ghare PM, Taylor RE (1969) Optimal redundancy for reliability in series systems. *Oper. Res.* 17(5):838–847.
Gordon R (1957) Optimum component redundancy for maximum system reliability. *Oper. Res.* 5(2):229–243.
Ha C, Kuo W (2006) Reliability redundancy allocation: An improved realization for nonconvex nonlinear programming problems. *Eur. J. Oper. Res.* 171(1):24–38.
Hsieh CC (2003) Optimal task allocation and hardware redundancy policies in distributed computing systems. *Eur. J. Oper. Res.* 147(2):430–447.
Huang HZ, Qu J, Zuo MJ (2009) Genetic-algorithm-based optimal apportionment of reliability and redundancy under multiple objectives. *IIE Trans.* 41(4):287–298.
Kim KO, Kuo W (2003) Percentile life and reliability as performance measures in optimal system design. *IIE Trans.* 35(12):1133–1142.
Konak A, Smith AE (2011) Efficient optimization of reliable two-node connected networks: A biobjective approach. *INFORMS J. Comput.* 23(3):430–445.
Kong X, Gao L, Ouyang H, Li S (2015) Solving the redundancy allocation problem with multiple strategy choices using a new simplified particle swarm optimization. *Reliability Engrg. System Safety* 144:147–158.
Kulturel-Konak S, Smith AE, Coit DW (2003) Efficiently solving the redundancy allocation problem using tabu search. *IIE Trans.* 35(6):515–526.
Kulturel-Konak S, Norman BA, Coit DW, Smith AE (2004) Exploiting tabu search memory in constrained problems. *INFORMS J. Comput.* 16(3):241–254.
Kuo W, Wan R (2007) Recent advances in optimal reliability allocation. *IEEE Trans. Systems Man Cybernetics-Part A Systems Humans* 37(2):143–156.
Li J, Huang Y, Li YF, Wang S (2023) Redundancy allocation under state-dependent distributional uncertainty of component lifetimes. *Production Oper. Management* 32(3):930–950.
Lin P, Leon B, Huang TC (1976) A new algorithm for symbolic system reliability analysis. *IEEE Trans. Reliability* 25(1):2–15.
Lodi A, Tramontani A (2013) Performance variability in mixed-integer programming. Topaloglu H, ed. *Theory Driven by Influential Applications*, INFORMS Tutorials in Operations Research (INFORMS, Catonsville, MD), 1–12.
Park YW (2020) MILP models for complex system reliability redundancy allocation with mixed components. *INFORMS J. Comput.* 32(3):600–619.
Prasad VR, Kuo W (2000) Reliability optimization of coherent systems. *IEEE Trans. Reliability* 49(3):323–330.
Prasad VR, Kuo W, Kim KO (2001) Maximization of a percentile life of a series system through component redundancy allocation. *IIE Trans.* 33(12):1071–1079.
Rai S, Veeraraghavan M, Trivedi KS (1995) A survey of efficient reliability computation using disjoint products approach. *Networks* 25(3):147–163.
Reihaneh M, Ardakan MA, Eskandarpour M (2022) An exact algorithm for the redundancy allocation problem with heterogeneous components under the mixed redundancy strategy. *Eur. J. Oper. Res.* 297(3):1112–1125.
Sallak M, Simon C, Aubry JF (2009) A reliability graph approach for availability and redundancy allocation: Application to safety instrumented systems. Technical report, Université de Technologie de Compiègne, Compiègne, France.
Sung CS, Cho YK (1999) Branch-and-bound redundancy optimization for a series system with multiple-choice constraints. *IEEE Trans. Reliability* 48(2):108–117.
Wang S, Li YF (2020) Distributionally robust design for redundancy allocation. *INFORMS J. Comput.* 32(3):620–640.
Wood RK (1985) A factoring algorithm using polygon-to-chain reductions for computing *K*-terminal network reliability. *Networks* 15(2):173–190.
Xu Y, Pi D (2019) A hybrid enhanced bat algorithm for the generalized redundancy allocation problem. *Swarm Evolutionary Comput.* 50:100562.
Yeh WC (2017) A new exact solution algorithm for a novel generalized redundancy allocation problem. *Inform. Sci.* 408:182–197.
Yeh WC, Lin YC, Chung YY (2010) Performance analysis of cellular automata Monte Carlo simulation for estimating network reliability. *Expert Systems Appl.* 37(5):3537–3544.
Yeh WC, Zhu W, Tan SY, Wang GG, Yeh YH (2022) Novel general active reliability redundancy allocation problems and algorithm. *Reliability Engrg. System Safety* 218:108167.
Zhao P, Zhang Y, Li L (2015) Redundancy allocation at component level versus system level. *Eur. J. Oper. Res.* 241(2):402–411.
Zhou X, Bai G, Tao J, Xu B (2023) An improved method to search all minimal paths in networks. *IEEE Trans. Reliability* 72(4):1420–1431.

Copyright of INFORMS Journal on Computing is the property of INFORMS: Institute for Operations Research & the Management Sciences and its content may not be copied or emailed to multiple sites or posted to a listserv without the copyright holder's express written permission. However, users may print, download, or email articles for individual use.